$\Huge Computational$ $\Huge Intelligence$ $\Huge Lab$

$\Huge Assignment$ $\Huge 1$

Jakub Jabłoński, Group 1 - Monday 11:30 - 13:00 GSI

1.1 Function

Consider the following Python class.

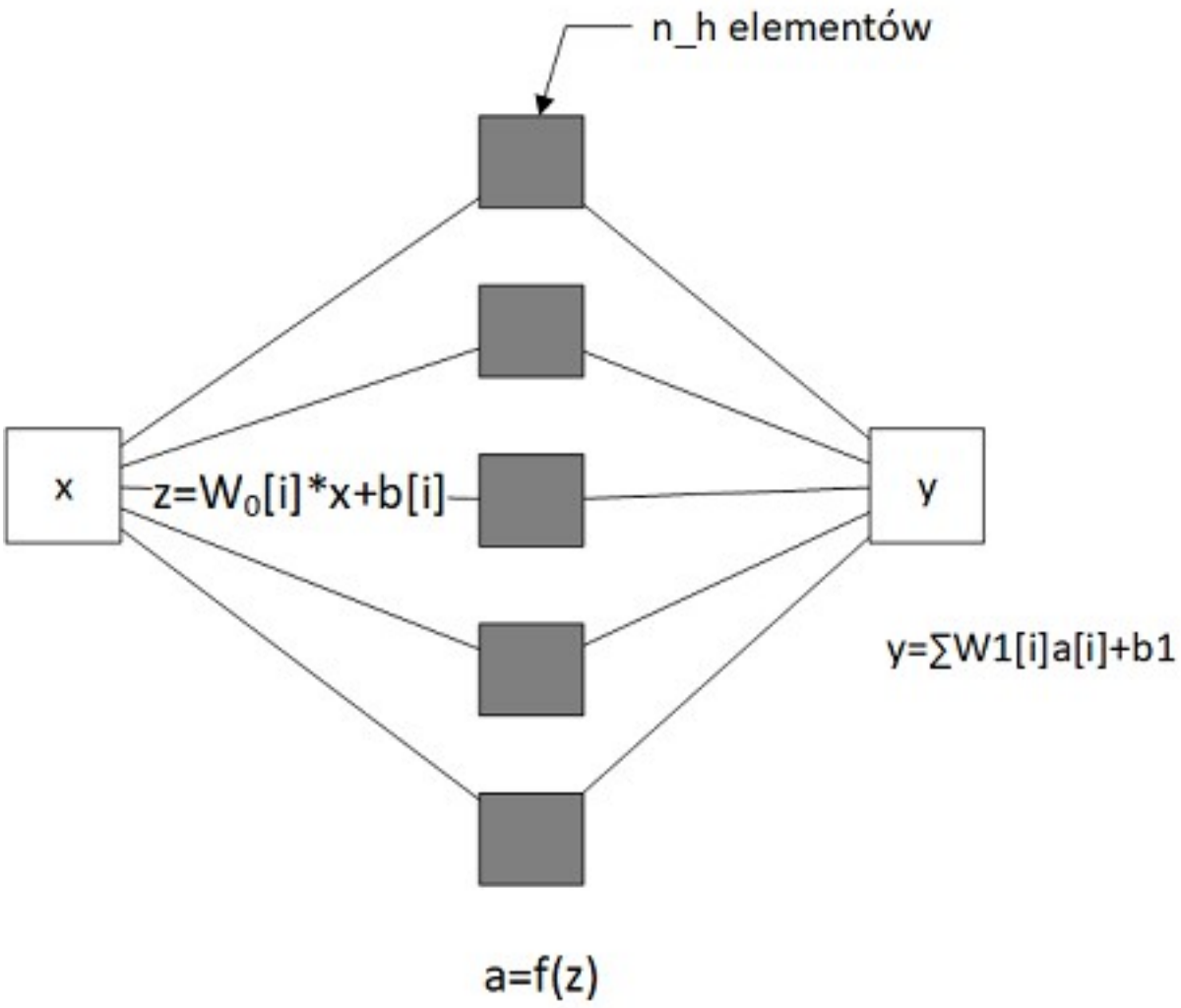
```
import numpy as np

class Function:
  def __init__(self,n_h,activation=lambda x : x):
    self.f=activation
    self.W0=np.random.randn(n_h,1)*np.sqrt(1/n_h)
    self.b0=np.zeros((n_h,1))
    self.W1=np.random.randn(1,n_h)*np.sqrt(1/n_h)
    self.b1=np.zeros((1,1))

  def __call__(self,x):
    z=self.W0*x+self.b0
    a  = self.f(z)
    y=np.dot(self.W1,a)+self.b1
    return y[0]

x=np.linspace(0,10,100)
f=Function(4)
y=f(x)

y
```

```
array([0.        , 0.07092369, 0.14184738, 0.21277107, 0.28369476,
       0.35461845, 0.42554215, 0.49646584, 0.56738953, 0.63831322,
       0.70923691, 0.7801606 , 0.85108429, 0.92200798, 0.99293167,
       1.06385536, 1.13477906, 1.20570275, 1.27662644, 1.34755013,
       1.41847382, 1.48939751, 1.5603212 , 1.63124489, 1.70216858,
       1.77309227, 1.84401596, 1.91493966, 1.98586335, 2.05678704,
       2.12771073, 2.19863442, 2.26955811, 2.3404818 , 2.41140549,
       2.48232918, 2.55325287, 2.62417657, 2.69510026, 2.76602395,
       2.83694764, 2.90787133, 2.97879502, 3.04971871, 3.1206424 ,
       3.19156609, 3.26248978, 3.33341348, 3.40433717, 3.47526086,
       3.54618455, 3.61710824, 3.68803193, 3.75895562, 3.82987931,
       3.900803  , 3.97172669, 4.04265038, 4.11357408, 4.18449777,
       4.25542146, 4.32634515, 4.39726884, 4.46819253, 4.53911622,
       4.61003991, 4.6809636 , 4.75188729, 4.82281099, 4.89373468,
       4.96465837, 5.03558206, 5.10650575, 5.17742944, 5.24835313,
```

```
       5.31927682, 5.39020051, 5.4611242 , 5.53204789, 5.60297159,
       5.67389528, 5.74481897, 5.81574266, 5.88666635, 5.95759004,
       6.02851373, 6.09943742, 6.17036111, 6.2412848 , 6.3122085 ,
       6.38313219, 6.45405588, 6.52497957, 6.59590326, 6.66682695,
       6.73775064, 6.80867433, 6.87959802, 6.95052171, 7.02144541])
```

Operations placed in the function call operator can be expressed as:

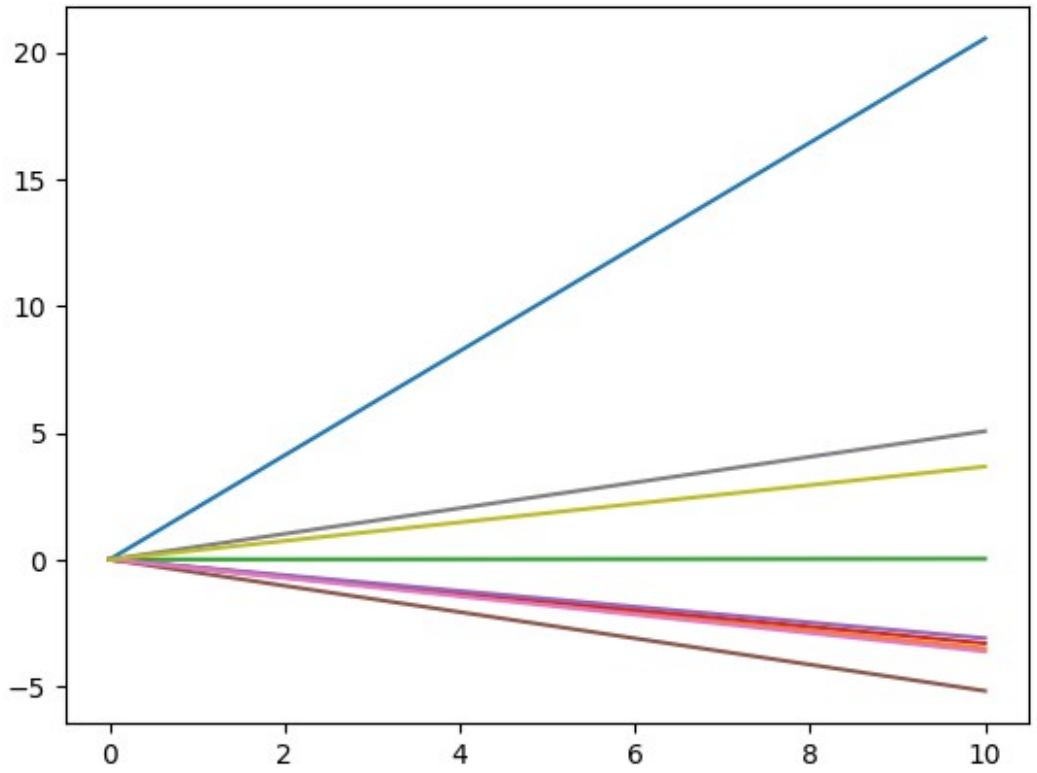
- $z = W_0 \cdot x + b_0$
- $a = f(z)$
- $y = W_1 * a + b_1$



TODO 1.1.1 Create function objects for various values of `n_h` and display their shapes. Use a for loop

```
import matplotlib.pyplot as plt
```

```
x=np.linspace(0,10,100)
for i  in range (1,10):
    plt.plot(x, Function(i)(x))
```

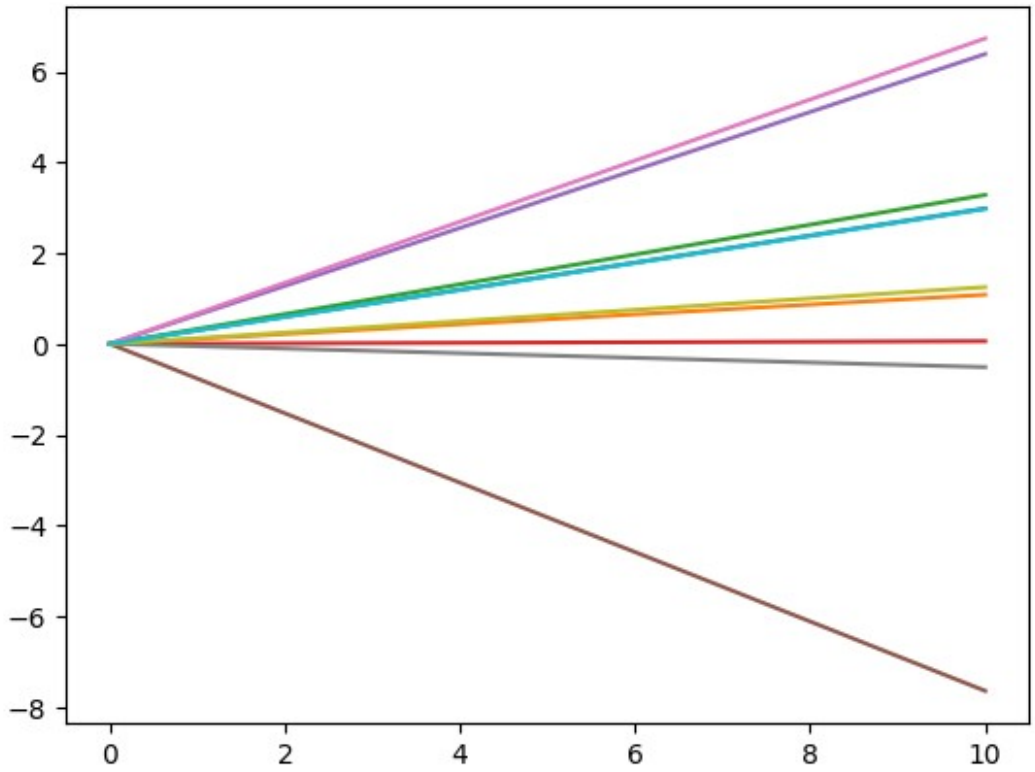


Run the following code.

Question: What are the shapes of function graphs? Why there are multiple plots?

Answer: Those are a linear functions. There are multiple of them because I ran it in loop

```
import matplotlib.pyplot as plt
for i in range(10):
  plt.plot(x,Function(5)(x))
```



TODO 1.1.2 Define two functions:

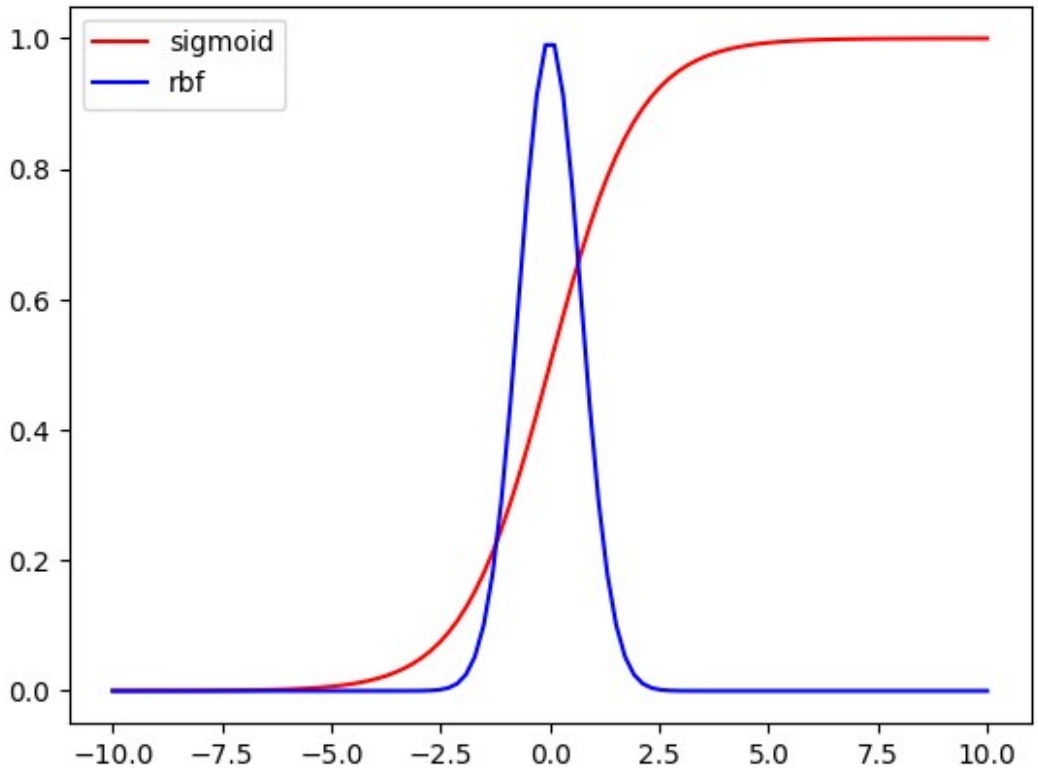
- $sigmoid(x)=\frac{1}{1+exp(-x)}$
- $rbf(x)=exp(-x^2)$

```
import numpy as np

def sigmoid(z):
  return 1/(1+np.exp(-z))

def rbf(z):
  return np.exp(-z**2)
```

then plot their graphs

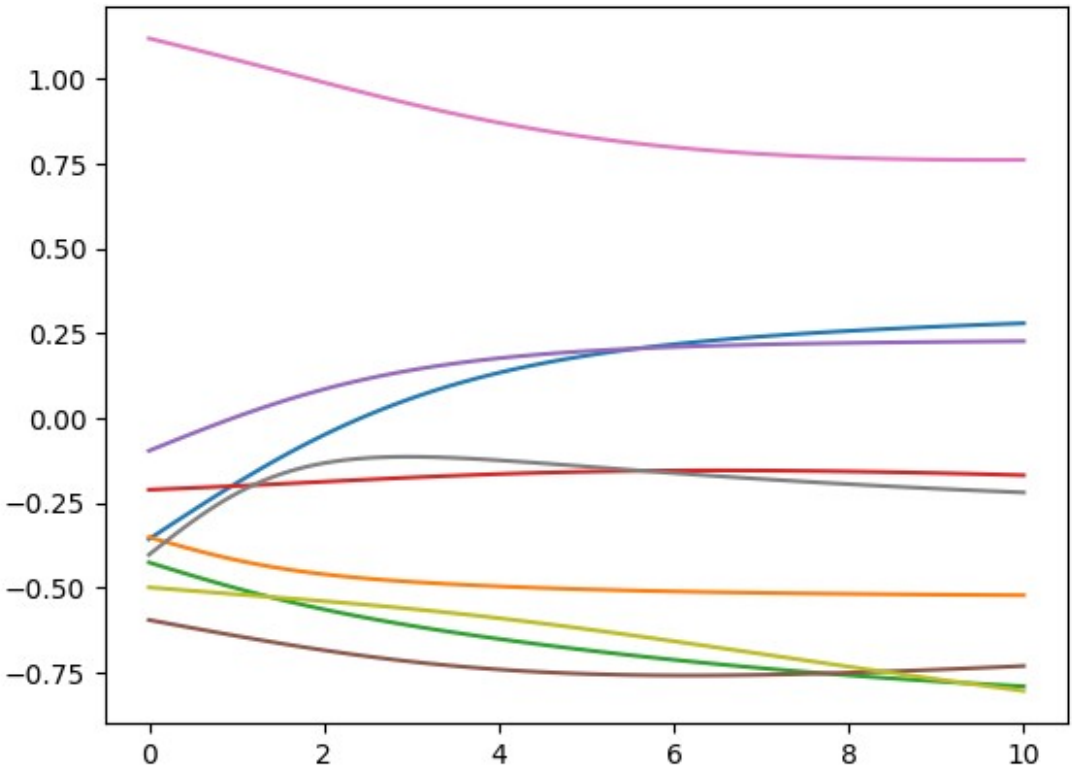
```
import matplotlib.pyplot as plt
x=np.linspace(-10,10,100)

plt.plot(x,sigmoid(x),c='r',label='sigmoid')
plt.plot(x,rbf(x),c='b',label='rbf')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7aa580736b10>
```



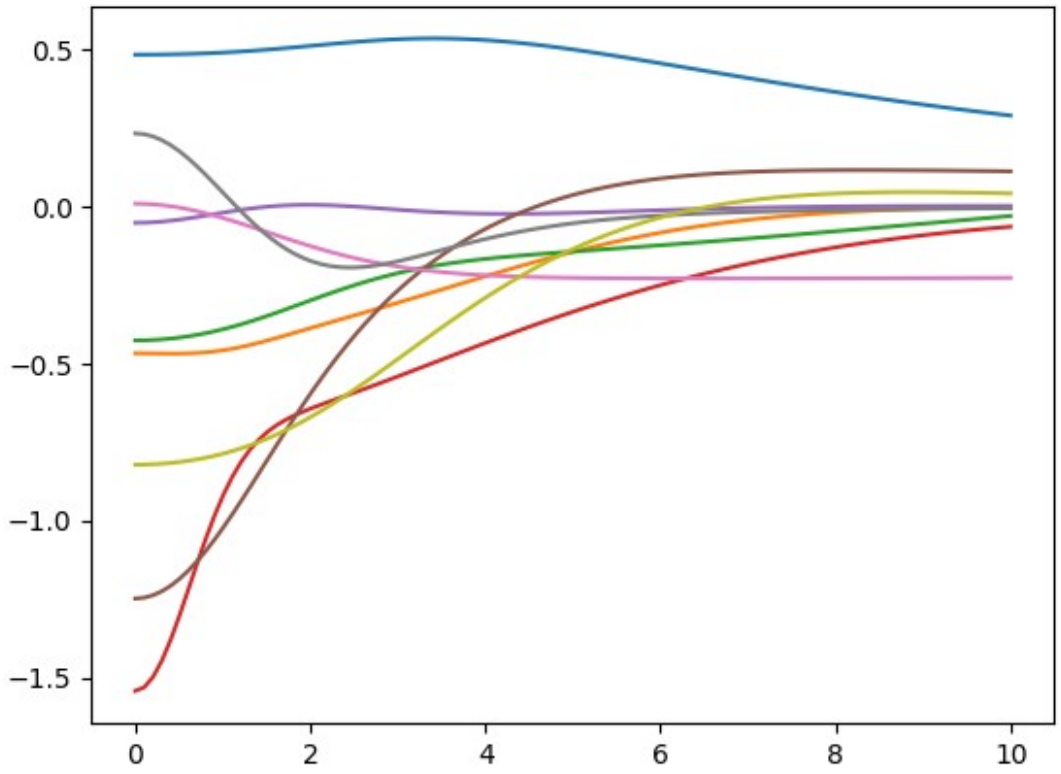
TODO 1.1.3 Display several function plots for activation = sigmoid

```
x=np.linspace(0,10,100)

f=Function(5, activation=sigmoid)

for i  in range (1,10):
    plt.plot(x, Function(5, activation=sigmoid)(x))
```



TODO 1.1.4 Display several function plots for activation = rbf

```
x=np.linspace(0,10,100)

f=Function(5, activation=sigmoid)

for i  in range (1,10):
    plt.plot(x, Function(5, activation=rbf)(x))
```



1.2 Implementation based on TensorFlow

```
import tensorflow as tf
print(tf.__version__)
```

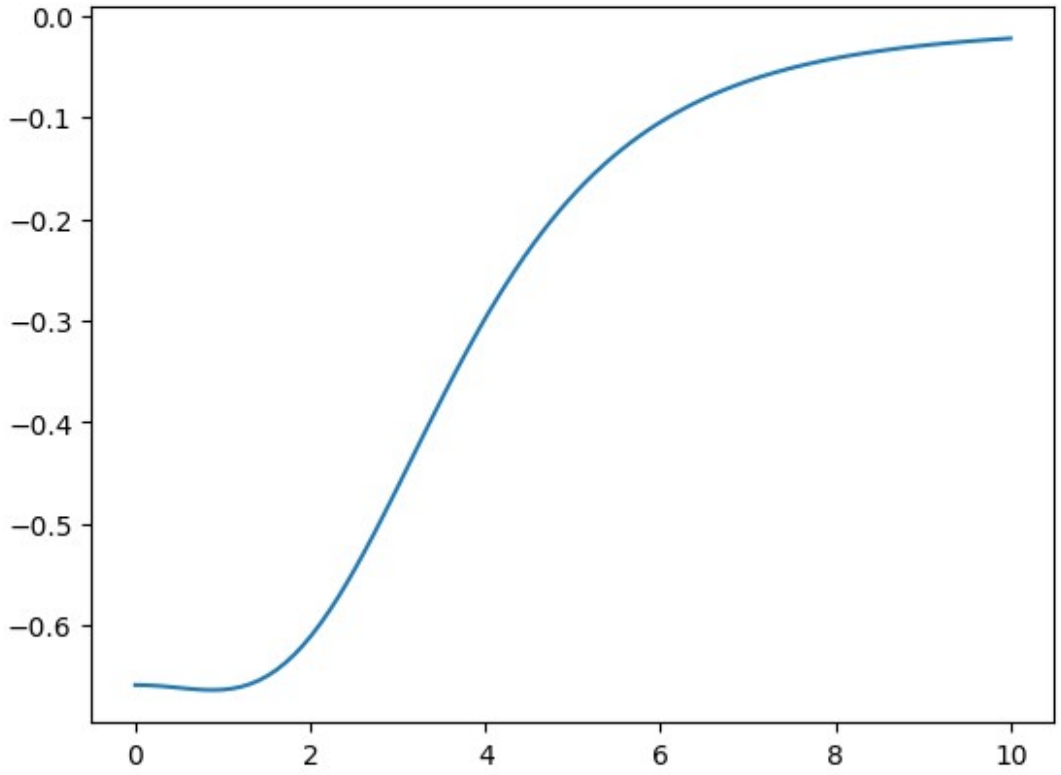
```
2.19.0
```

```
import tensorflow as tf


class Function:
  def __init__(self,n_h,activation = lambda x:x):
    self.f = activation
    self.W0=tf.Variable(np.random.randn(n_h,1)*np.sqrt(1/n_h))
    self.b0=tf.Variable(np.zeros((n_h,1)))
    self.W1=tf.Variable(np.random.randn(1,n_h)*np.sqrt(1/n_h))
    self.b1=tf.Variable(np.zeros((1,1)))


  def __call__(self,x):
    z=self.W0*x+self.b0
    # a=sigmoid(z)
    a=self.f(z)
    y=tf.matmul(self.W1,a)+self.b1
    return y
```

Run the cell below several times. Each time the function shape changes.

```
import numpy as np
x=np.linspace(0,10,100)
f=Function(10,activation=rbf)
y=f(x)
plt.plot(x,y.numpy()[0])
```

```
[<matplotlib.lines.Line2D at 0x7aa57bf13860>]
```



How to fit the model to a given function?

We need

1. A measure to evaluate model fitness
2. A loss function to find the optimal model
3. Loss function may be identical to measure (but does not have to)
4. An optimization procedure that minimizes the loss

TODO 1.2.1 Analyze the code in the cell below and complete the code of MSE function. MSE means Mean Squared Error

```
a = tf.Variable([1.0,2.0,3.0,4.5])
b = tf.Variable([1.0,2.1,3.8,4.0])
e = (a-b)**2

def tf_MSE(y_true,y_pred):
  e = (y_true - y_pred)**2
  return tf.math.reduce_sum(e)/e.shape[0]

print(e)
mse=sum(e[i] for i in range(e.shape[0]))/e.shape[0]
print(mse)
```

```
tf.Tensor([0.         0.00999998 0.6399999  0.25      ], shape=(4,),
dtype=float32)
tf.Tensor(0.22499998, shape=(), dtype=float32)
```

```
def tf_MSE(y_true,y_pred):
  e = (y_true - y_pred)**2
  return tf.math.reduce_sum(e)/e.shape[0]
```

TODO 1.2.2 Rewrite sigmoid and rbf functions using TensorFlow

```
def tf_sigmoid(x):
  return 1/(1+tf.exp(-x))

def tf_rbf(x):
  return tf.exp(-tf.square(x))
```

The fit method

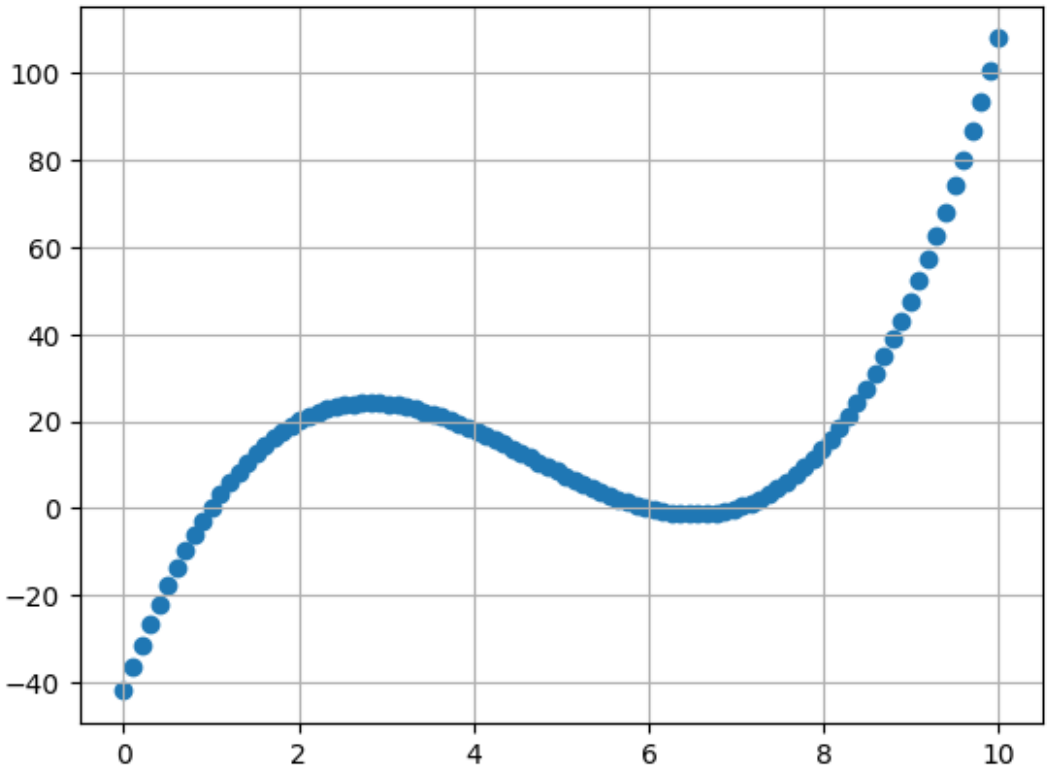
- Input: x and y
- Iterates multiple times (parameter epoch)
- In each iteration
  - Calculates y_pred = model(x)
  - Computes loss function
  - Computes gradient of loss function with respect to weights
  - Updates weights, basicaly according to the formula $W = W - gradient * learning_rate$.
  - Actually uses an optimizer that performs this in a smarter way

```
import tensorflow as tf


class Function:
  def __init__(self,n_h,activation = lambda x:x):
    self.f = activation
    self.W0=tf.Variable(np.random.randn(n_h,1)*0.01)
    self.b0=tf.Variable(np.zeros((n_h,1)))
    self.W1=tf.Variable(np.random.randn(1,n_h)*0.01)
```

```
    self.b1=tf.Variable(np.zeros((1,1)))


  def __call__(self,x):
    z=self.W0*x+self.b0
    a=self.f(z)
    y=tf.matmul(self.W1,a)+self.b1
    return y


  def fit(self,x,y,epochs=10,optimizer =
tf.keras.optimizers.RMSprop()):
    for i in range(epochs):
      with tf.GradientTape() as tape:
        y_pred=self(x)
        loss = tf_MSE(y_pred,y)
        # print(loss)
        variables=(self.W0,self.b0,self.W1,self.b1)
        gradients = tape.gradient(loss, variables)
        # print(gradients)
        optimizer.apply_gradients(zip(gradients, variables))
```

We will try to fit our model (Function class) to the polynomial $y=(x-1)(x-6)(x-7)$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0,10,100)
y = (x-1)*(x-6)*(x-7)

plt.scatter(x,y)
plt.grid()
```

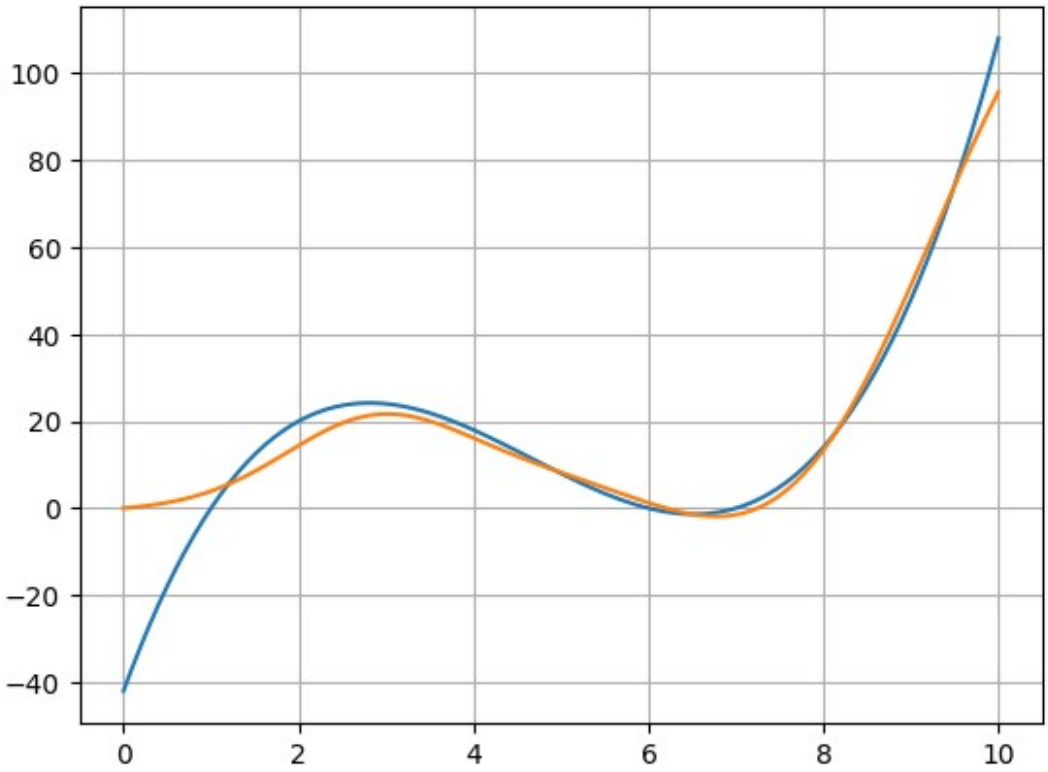


Although the problem is super-easy for methods numerical analysis, using this approach is a little bit hard. We need many hidden units and iterations... (execution abot 90 sec)

TODO 1.2.3 Create a model (Function) object passing as parameters 50 hidden units and rbf activation function. Fit the model setting number of iterations to 5000.

```
f=Function(50, activation=tf_rbf)
f.fit(x,y, epochs=5000, optimizer = tf.keras.optimizers.RMSprop())

plt.plot(x,y)
plt.grid()

y_pred=f(x)
plt.plot(x,y_pred.numpy()[0])
```

```
[<matplotlib.lines.Line2D at 0x7aa57b6d9280>]
```



Hyperparameters

- n_h (number of hidden neurons) controls the model complexity
- activation function - influences the model performance
- epochs - controls number of iterations (influences the learning algorithm)

1.3 Implementation based on Pytorch

TODO 1.3.1 Write functions `torch_rbf` and `torch_MSE`

```
import torch

def torch_rbf(x):
    return torch.exp(-torch.pow(x, 2))

x = torch.tensor([1.0, 2.0])

print(torch_rbf(x))
```

```
tensor([0.3679, 0.0183])
```

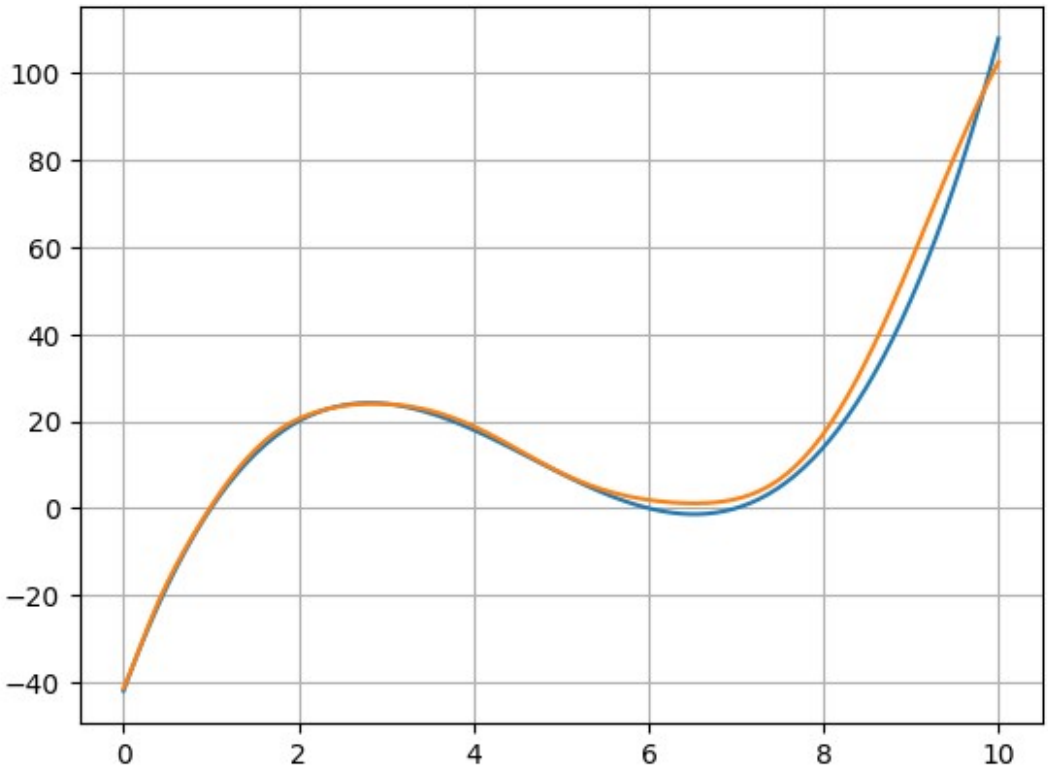
```
def torch_MSE(y_pred, y_true):
    e = torch.pow((y_true - y_pred),2)
    return e.mean()
```

```
import torch
import torch.nn as nn
import torch.optim as optim



class Function(nn.Module):
    def __init__(self, n_h, activation=lambda x: x):
        super(Function, self).__init__()
        self.f = activation

        # nn.Parameter (analog of tf.Variable)
        self.W0 = nn.Parameter(torch.randn(n_h, 1) * 0.1)
        self.b0 = nn.Parameter(torch.zeros(n_h, 1))
        self.W1 = nn.Parameter(torch.randn(1, n_h) * 0.1)
        self.b1 = nn.Parameter(torch.zeros(1, 1))

    def forward(self, x):
        z = self.W0 @ x + self.b0   # computes: W0 * x + b0
        a = self.f(z)               # activation
        y = self.W1 @ a + self.b1   # computes: W1 * a + b1
        return y


    def fit(self, x, y, epochs=10, optimizer=None):
        if optimizer is None:
            optimizer = optim.RMSprop(self.parameters())

        for i in range(epochs):
            optimizer.zero_grad()         # sets gradients to 0
            y_pred = self.forward(x)
            loss = torch_MSE(y_pred, y)
            loss.backward()               # gradient computation
            optimizer.step()              # applying gradients to
weights
            # print(loss.item())



model = Function(n_h=50, activation=torch_rbf)

x = np.linspace(0,10,100)
y = (x-1)*(x-6)*(x-7)

x_t = torch.tensor(x, dtype=torch.float32).unsqueeze(0)
y_t = torch.tensor(y, dtype=torch.float32).unsqueeze(0)
model.fit(x_t, y_t, epochs=5000)

y_pred = model(x_t)
print(y_pred)
```

```
tensor([[-41.3688, -36.5015, -31.2256, -26.0497, -21.2489, -16.8913, -12.9192,
          -9.2332,  -5.7492,  -2.4224,   0.7551,   3.7665,   6.5855,   9.1862,
          11.5494,  13.6650,  15.5324,  17.1585,  18.5566,  19.7443,  20.7414,
          21.5687,  22.2462,  22.7923,  23.2230,  23.5517,  23.7884,  23.9407,
          24.0129,  24.0073,  23.9241,  23.7620,  23.5186,  23.1913,  22.7776,
          22.2753,  21.6837,  21.0031,  20.2359,  19.3864,  18.4612,  17.4688,
          16.4199,  15.3271,  14.2042,  13.0664,  11.9290,  10.8074,   9.7163,
           8.6692,   7.6777,   6.7513,   5.8971,   5.1194,   4.4203,   3.7993,
           3.2539,   2.7803,   2.3736,   2.0287,   1.7413,   1.5078,   1.3269,
           1.1993,   1.1286,   1.1214,   1.1874,   1.3393,   1.5925,   1.9649,
           2.4759,   3.1465,   3.9979,   5.0512,   6.3264,   7.8417,   9.6132,
          11.6537,  13.9725,  16.5752,  19.4631,  22.6328,  26.0768,  29.7829,
          33.7348,  37.9119,  42.2902,  46.8419,  51.5366,  56.3415,  61.2217,
          66.1413,  71.0633,  75.9507,  80.7667,  85.4753,  90.0419,  94.4334,
          98.6191, 102.5705]], grad_fn=<AddBackward0>)
```

```
plt.plot(x,y)
plt.grid()

plt.plot(x,y_pred.detach().numpy()[0])
```

```
[<matplotlib.lines.Line2D at 0x7aa4467b00b0>]
```



1.4 TensorFlow: neural network model

Analogous model can be built using components of tensorflow.keras library.

- **Advantage** the computations are converted to form a *computational graph* that can be executed much faster. Also on GPU. This is done with `compile` method.

```
from keras import models
from keras import layers

def build_model(n_h):
  model = models.Sequential()
  model.add(layers.Input(shape=(1,)))
  model.add(layers.Dense(n_h, activation=tf_rbf))
  # model.add(layers.Dense(n_h, activation=tf_rbf))
  model.add(layers.Dense(1))
  model.compile(optimizer='rmsprop', loss='mse',
metrics=['mse','mae'])
  return model
```

TODO 1.4.1 Create a model with 50 hidden units and call fit function setting number of epochs 5000 and batch_size (another hyperparameter) to 100

```
import numpy as np
x = np.linspace(0,10,100)
y = (x-1)*(x-6)*(x-7)

model = build_model(100)
model.summary()
history = model.fit(
    x, y,
    epochs=5000,
    batch_size=100,
    verbose=0
)
```

```
Model: "sequential"
```

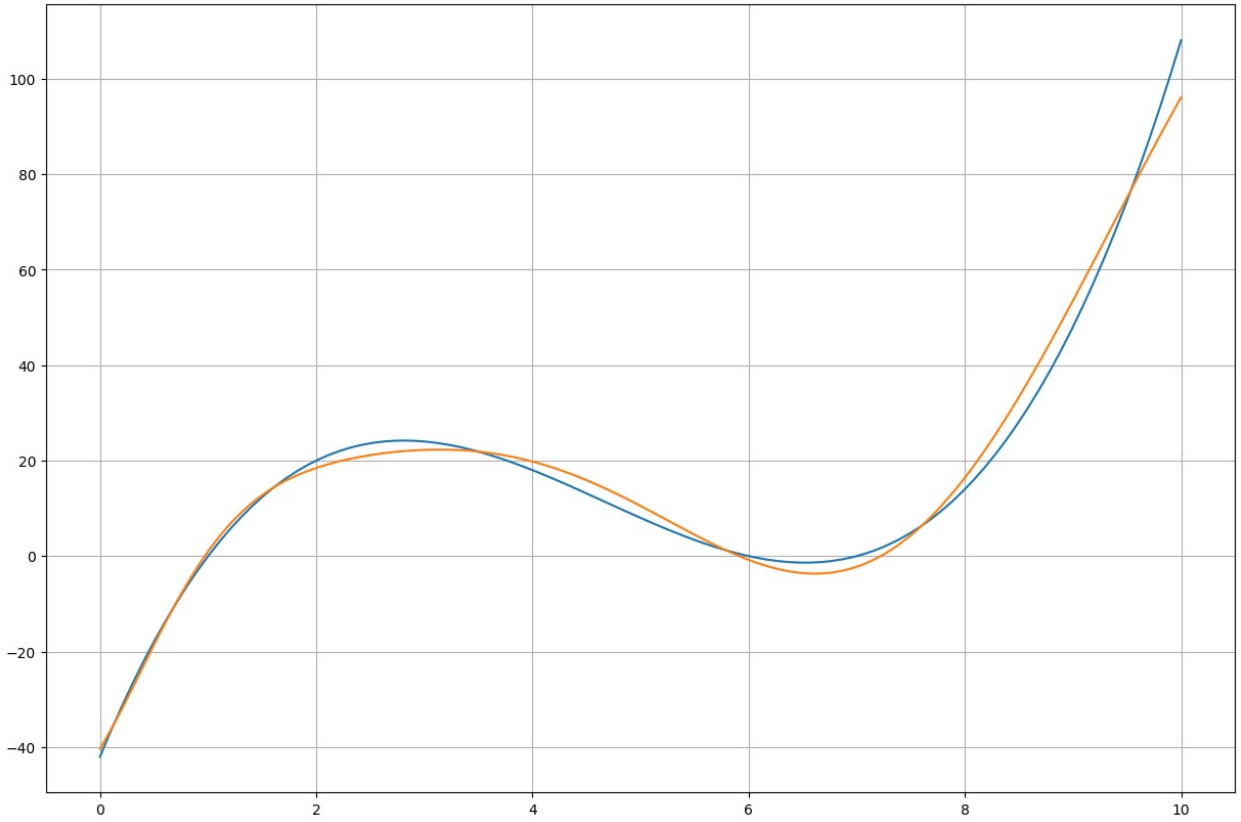
```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ dense (Dense)                   │ (None, 100)            │           200 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_1 (Dense)                 │ (None, 1)              │           101 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 301 (1.18 KB)

 Trainable params: 301 (1.18 KB)

 Non-trainable params: 0 (0.00 B)
```

Check plots of original and fit curves

```
plt.figure(figsize=(15,10))

plt.plot(x,y)
plt.grid()

y_pred=model.predict(x)
plt.plot(x,y_pred)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step

[<matplotlib.lines.Line2D at 0x7aa44647f2f0>]
```

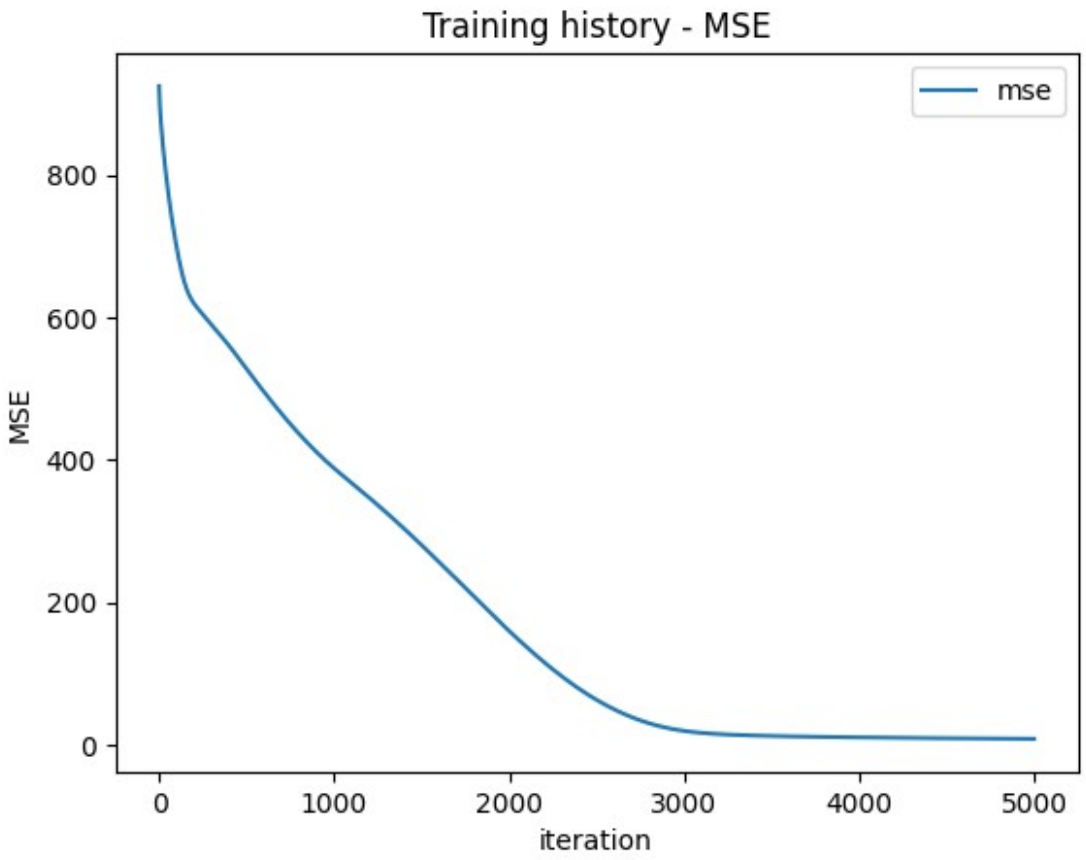


TODO 1.4.2 Repat the above steps changing hyperparameters to get a good fit

During training some data are collected. We may display various measures residing in the training history

```
import matplotlib.pyplot as plt

plt.title('Training history - MSE')
plt.plot(history.history['mse'],label='mse')
plt.xlabel('iteration')
plt.ylabel('MSE')
plt.legend()

# history.history['mse']
```

```
<matplotlib.legend.Legend at 0x7aa4466a59a0>
```



1.5 PyTorch: Neural network model

In PyTorch activation function is a separate layer

Information on loss should be explicitely collected

```
history = {'loss': []}

for epoch in range(epochs):
    ...
    history['loss'].append(loss.item())
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# --- RBF Layer ---
class RBF(nn.Module):
    def forward(self, x):
```

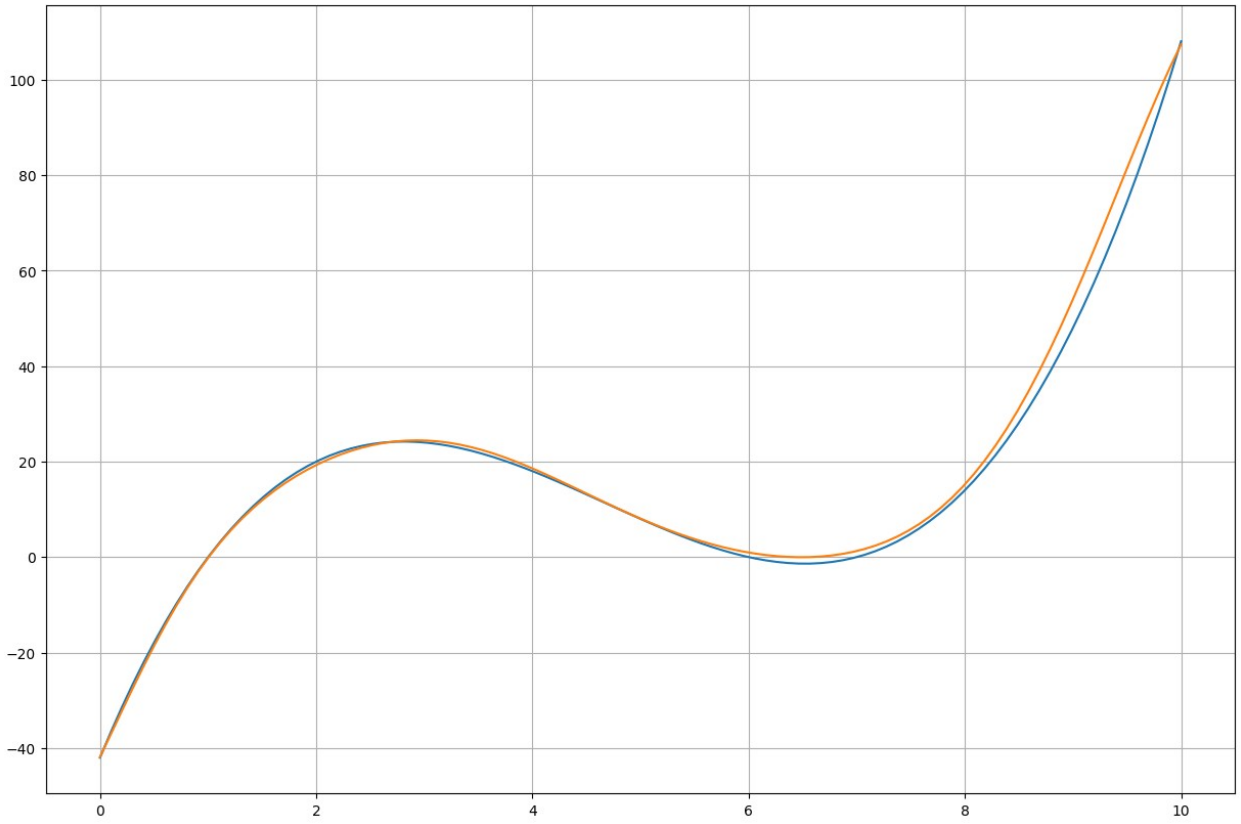
```
        return torch.exp(-x**2)


class SimpleRBFNN(nn.Module):
    def __init__(self, n_h):
        super().__init__()
        self.hidden = nn.Linear(1, n_h)
        self.rbf = RBF()
        self.out = nn.Linear(n_h, 1)

    def forward(self, x):
        x = self.hidden(x)
        x = self.rbf(x)
        x = self.out(x)
        return x

# datatype and shape conversion
x = np.linspace(0, 10, 100).reshape(-1, 1).astype(np.float32)
y = ((x - 1)*(x - 6)*(x - 7)).astype(np.float32)

x_tensor = torch.from_numpy(x)
y_tensor = torch.from_numpy(y)

# --- Model, optimizer, loss ---
model = SimpleRBFNN(n_h=50)
optimizer = optim.RMSprop(model.parameters())
loss_fn = nn.MSELoss()

# --- Trening ---
epochs = 5000
history = {'loss': []}

for epoch in range(epochs):
    optimizer.zero_grad()
    y_pred = model(x_tensor)
    loss = loss_fn(y_pred, y_tensor)
    loss.backward()
    optimizer.step()

    history['loss'].append(loss.item())

    if (epoch+1) % 1000 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")

# --- Sprawdzenie wyników ---
with torch.no_grad():
    y_pred = model(x_tensor)


Epoch 1000, Loss: 14.157632
Epoch 2000, Loss: 4.978427
```

```
Epoch 3000, Loss: 4.520299
Epoch 4000, Loss: 4.088841
Epoch 5000, Loss: 5.506890
```

prior to conversion to numpy, gradients should be detached ...

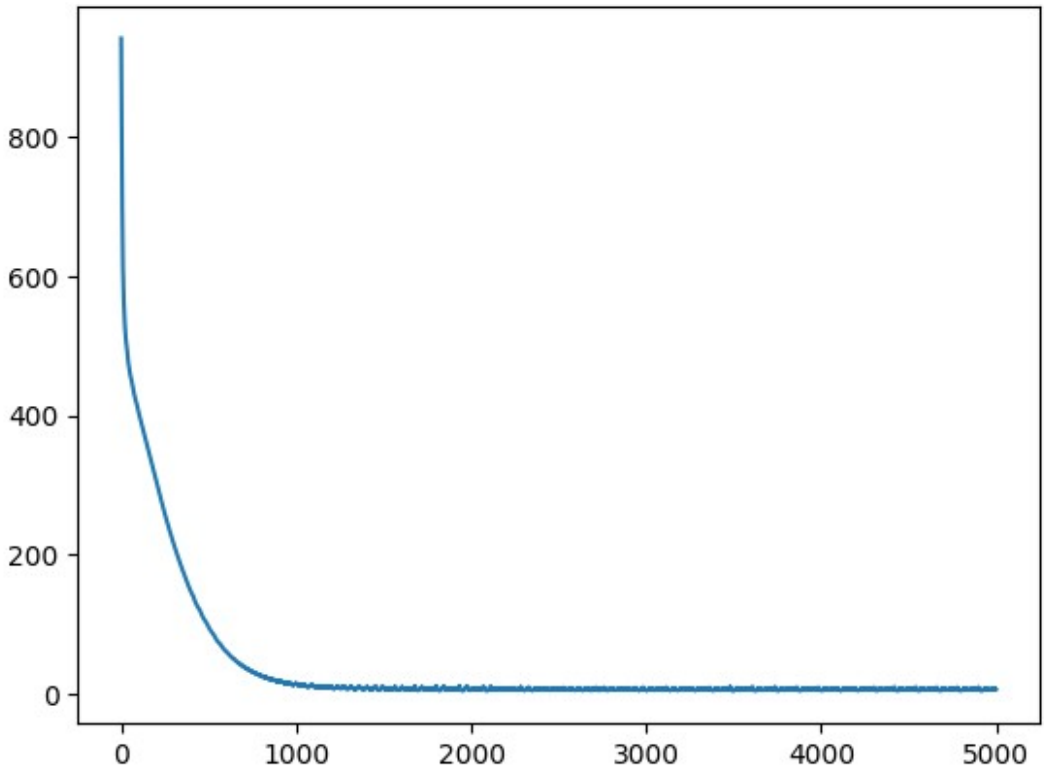
```
import matplotlib.pyplot as plt
plt.figure(figsize=(15,10))

plt.plot(x,y)
plt.grid()

y_pred=model(x_tensor).detach().numpy()
plt.plot(x,y_pred)
```

```
[<matplotlib.lines.Line2D at 0x7aa4462c8f50>]
```




```
plt.plot(history['loss'])
```

```
[<matplotlib.lines.Line2D at 0x7aa44507d8e0>]
```

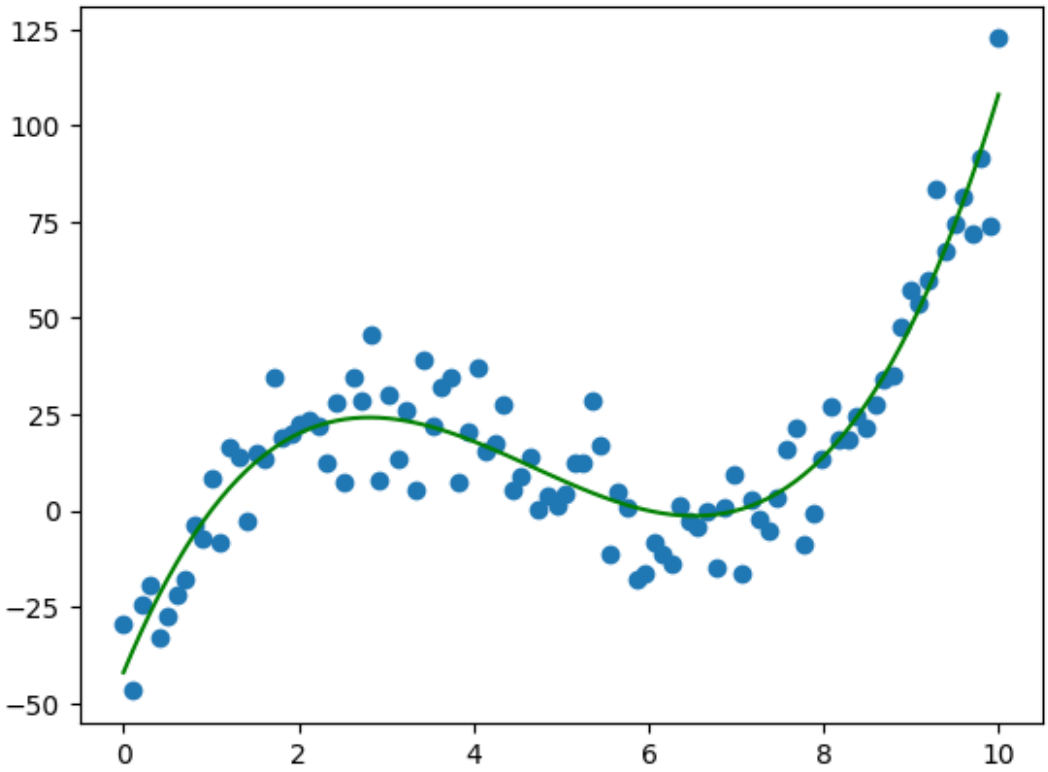


1.6 More realistic problem

The task of perfectly fitting a known function is very rare.

- It is rather assumed that we have data that originate from a true underlaying function with a noise $y=f(x)+\varepsilon$
- It is also often assumed that $\varepsilon \sim N(0,\sigma)$

```
from keras import models
from keras import layers
import numpy as np
import matplotlib.pyplot as plt

n_size=100
x = np.linspace(0,10,n_size)
y = (x-1)*(x-6)*(x-7)+np.random.normal(0,10,n_size)

plt.scatter(x,y)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')


[<matplotlib.lines.Line2D at 0x7aa4450d51f0>]
```



1.6.1 Tensorflow

TODO 1.6.1 Fit the model to this DATA using the best hyperparameters obtained before

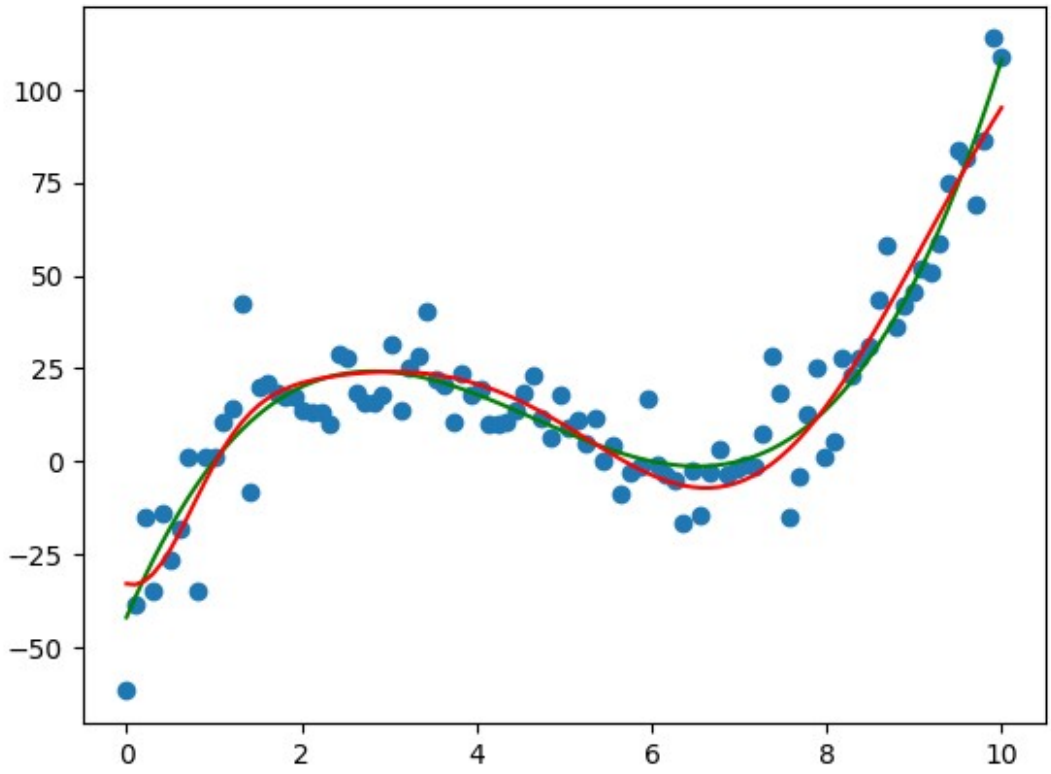
```
model = build_model(n_h=100)
history = model.fit(
    x,y,
    epochs=5000,
    batch_size=100,
    verbose=0
)
```

TODO 1.6.2 Plot the scattered data, true function in green and predictions in red

```
n_size=100
x = np.linspace(0,10,n_size)
y = (x-1)*(x-6)*(x-7)+np.random.normal(0,10,n_size)
y_predict=model.predict(x)

plt.scatter(x,y)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
plt.plot(x,y_predict, color='r')
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```

```
[<matplotlib.lines.Line2D at 0x7aa444ff2930>]
```



1.6.2 Pytorch

TODO 1.6.3 Write analogous code using PyTorch library. Select number of epochs providing the best results (no underfitting / overfitting. Plot the results.

Collect information on loss and display it.

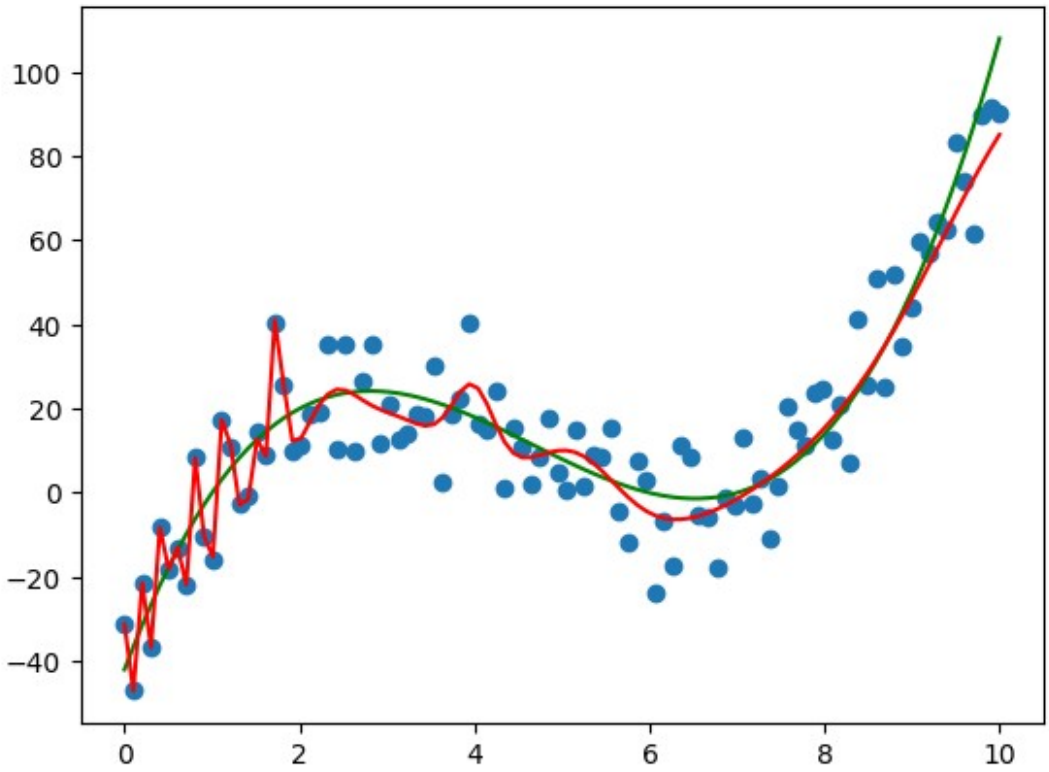
```
n_size=100
x = np.linspace(0,10,n_size).reshape(-1,1).astype(np.float32)
y = (x-1)*(x-6)*(x-7)+np.random.normal(0,10,
(n_size,1)).astype(np.float32)

x_tensor = torch.from_numpy(x)
y_tensor = torch.from_numpy(y)

# --- Model, optimizer, loss ---
model = SimpleRBFNN(n_h=100)
optimizer = optim.RMSprop(model.parameters())
loss_fn = nn.MSELoss()

# --- Trening ---
epochs = 10000
history = {'loss': []}
```

```python
for epoch in range(epochs):
    optimizer.zero_grad()
    y_pred = model(x_tensor)
    loss = loss_fn(y_pred, y_tensor)
    loss.backward()
    optimizer.step()

    history['loss'].append(loss.item())

    if (epoch+1) % 1000 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")

# --- Sprawdzenie wyników ---
with torch.no_grad():
    y_pred = model(x_tensor)
```

```
Epoch 1000, Loss: 96.092796
Epoch 2000, Loss: 83.853607
Epoch 3000, Loss: 80.633949
Epoch 4000, Loss: 76.593254
Epoch 5000, Loss: 74.374435
Epoch 6000, Loss: 73.523849
Epoch 7000, Loss: 72.522362
Epoch 8000, Loss: 72.517120
Epoch 9000, Loss: 72.055984
Epoch 10000, Loss: 71.284500
```

```python
# Example 1
y_pred=model(x_tensor).detach().numpy()

plt.scatter(x,y)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
plt.plot(x,y_pred, color='r')
```

```
[<matplotlib.lines.Line2D at 0x7aa444bf4c80>]
```

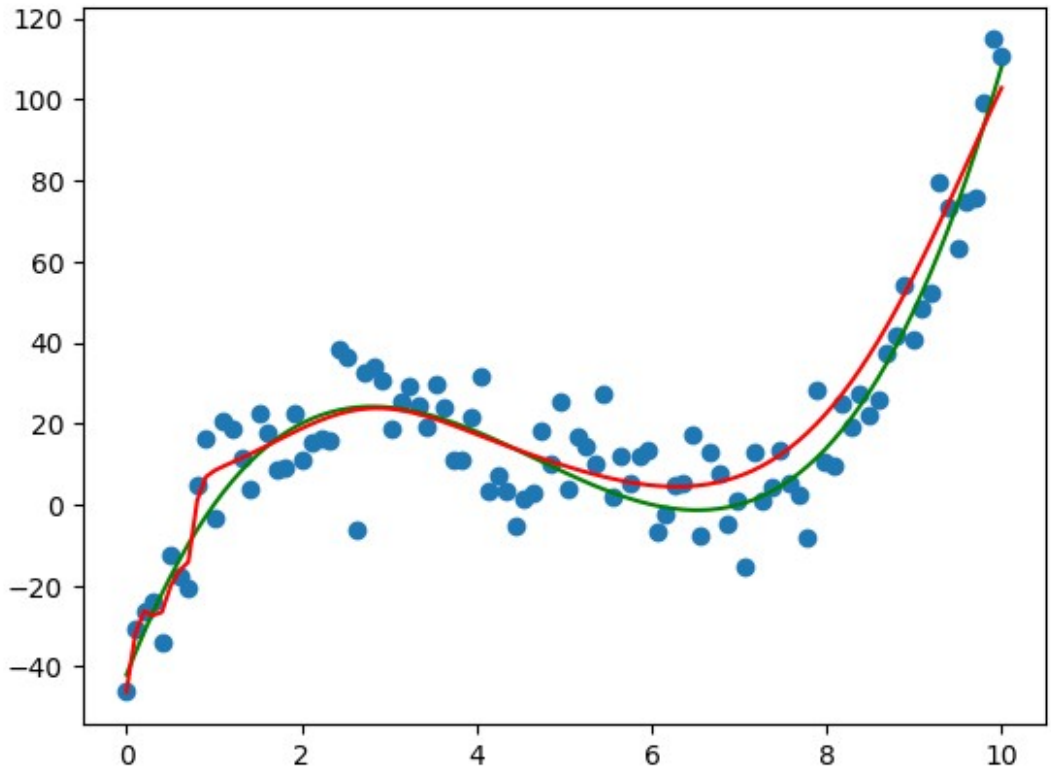



```python
# Example 2
model = SimpleRBFNN(n_h=100)
optimizer = optim.RMSprop(model.parameters())
loss_fn = nn.MSELoss()

# --- Trening ---
epochs = 1000
history = {'loss': []}

for epoch in range(epochs):
    optimizer.zero_grad()
    y_pred = model(x_tensor)
    loss = loss_fn(y_pred, y_tensor)
    loss.backward()
    optimizer.step()

    history['loss'].append(loss.item())

    if (epoch+1) % 1000 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")

# --- Sprawdzenie wyników ---
with torch.no_grad():
    y_pred = model(x_tensor)

y_pred=model(x_tensor).detach().numpy()
```

```
plt.scatter(x,y)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
plt.plot(x,y_pred, color='r')
```

```
Epoch 1000, Loss: 101.188515
```

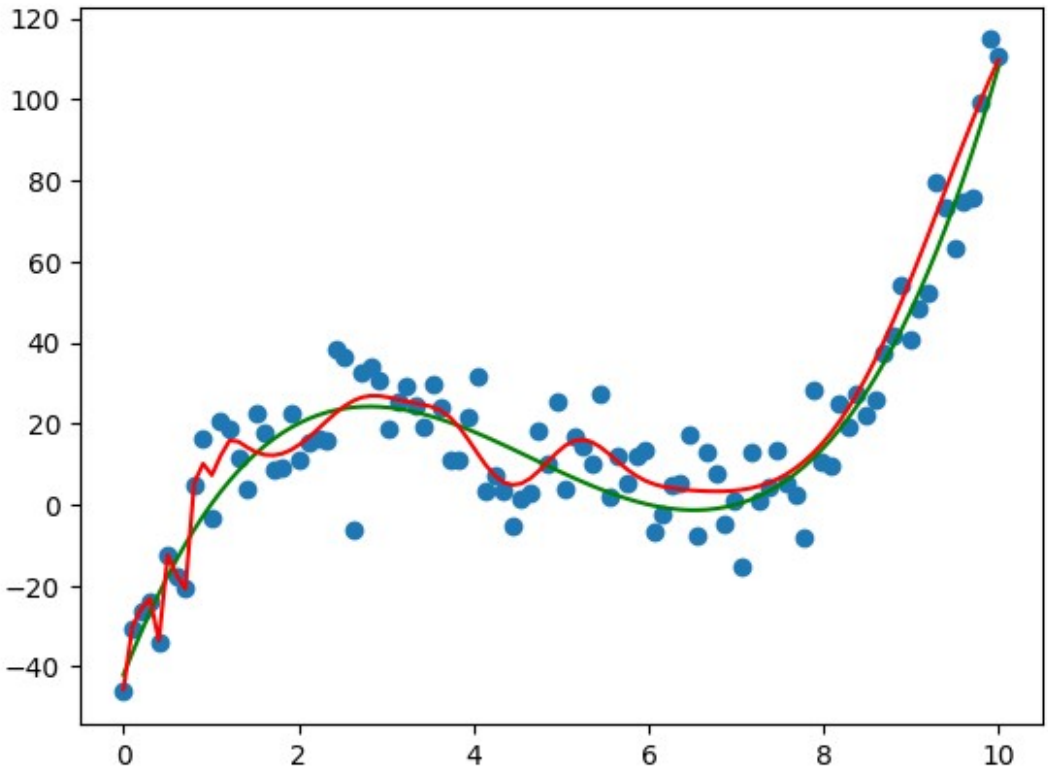
```
[<matplotlib.lines.Line2D at 0x7aa43c6e7530>]
```




```
# Example 3
model = SimpleRBFNN(n_h=100)
optimizer = optim.RMSprop(model.parameters())
loss_fn = nn.MSELoss()

# --- Trening ---
epochs = 5000
history = {'loss': []}

for epoch in range(epochs):
    optimizer.zero_grad()
    y_pred = model(x_tensor)
    loss = loss_fn(y_pred, y_tensor)
    loss.backward()
    optimizer.step()
```

```
    history['loss'].append(loss.item())

    if (epoch+1) % 1000 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")

# --- Sprawdzenie wyników ---
with torch.no_grad():
    y_pred = model(x_tensor)

y_pred=model(x_tensor).detach().numpy()

plt.scatter(x,y)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
plt.plot(x,y_pred, color='r')
```

```
Epoch 1000, Loss: 95.272537
Epoch 2000, Loss: 83.903702
Epoch 3000, Loss: 81.204292
Epoch 4000, Loss: 78.837868
Epoch 5000, Loss: 77.087029

[<matplotlib.lines.Line2D at 0x7aa43c4322a0>]
```



1.7 Validating model - training and testing

Typical ML workflow includes training the model and testing its performance on unseen data.

- **Why** - to control and assess generalization error which may result from
  - underfitting - the model is to simple or not trained enough
  - overfitting - the model is too complex, matches perfectly the training data (see part of the plot on the left)

We will split the data into two subsets

```
from sklearn.model_selection import train_test_split

x = np.linspace(0,10,n_size)
y = (x-1)*(x-6)*(x-7)+np.random.normal(0,10,n_size)

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=123)
```

1.7.1 TensorFlow

TODO 1.7.1 Fit the TensorFlow model using x_train and y_train, set the parameter `validation_data=(x_test, y_test)`

Warning: training lasts up to 250 sec

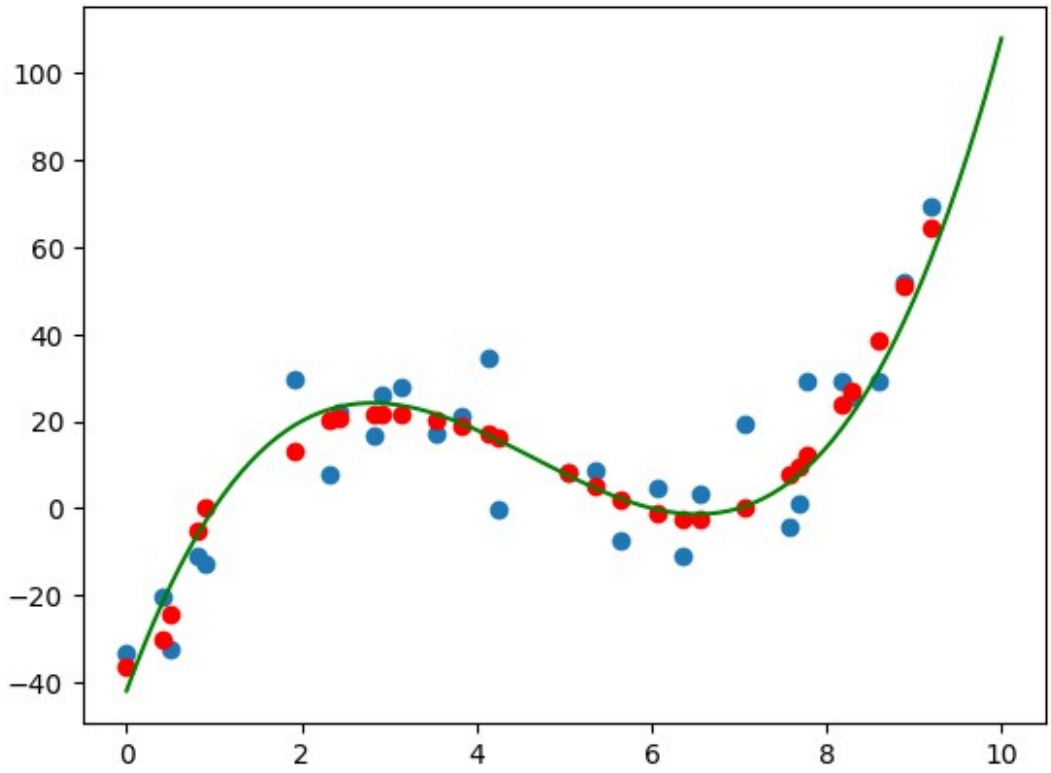
```
model = build_model(n_h=100)
history = model.fit(
    x_train, y_train,
    epochs=10000,
    batch_size=100,
    validation_data=(x_test, y_test),
    verbose=0
)
```

We will display true function, noisy data and predictions

```
plt.scatter(x_test,y_test)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
y_pred=model.predict(x_test)
plt.scatter(x_test,y_pred,color='r')
# plt.plot(x_test,y_pred,color='r')
```

```
WARNING:tensorflow:5 out of the last 9 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dist
ributed at 0x7aa443c94ae0> triggered tf.function retracing. Tracing is
expensive and the excessive number of tracings could be due to (1)
creating @tf.function repeatedly in a loop, (2) passing tensors with
different shapes, (3) passing Python objects instead of tensors. For
```

```
(1), please define your @tf.function outside of the loop. For (2),
@tf.function has reduce_retracing=True option that can avoid
unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more
details.

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 192ms/step

<matplotlib.collections.PathCollection at 0x7aa444d56d80>
```

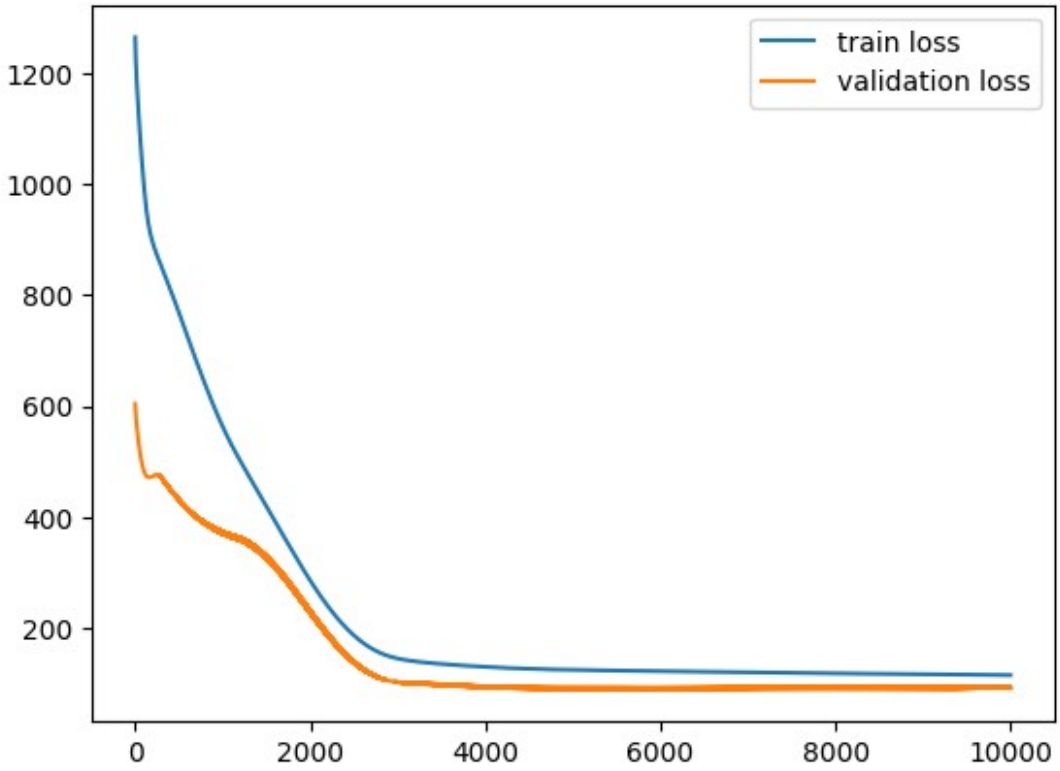


Lets peek what is the content of the history...

```
for k in history.history:
  print(k)

loss
mae
mse
val_loss
val_mae
val_mse
```

TODO 1.7.2 Display loss (training loss) and val_loss (validation loss on the test set)

```
plt.plot(history.history['loss'], label="train loss")
plt.plot(history.history['val_loss'], label="validation loss")
plt.legend()
plt.show()
```



1.7.2 PyTorch

TODO 1.7.3 Perform analogous task using PyTorch.

While evaluating model on test data you should not compute gradients. The typical code snippet is:

```
    # Within the traing loop
    model.eval()
    with torch.no_grad():
        y_val_pred = model(X_test)
        val_loss = loss_fn(y_val_pred, y_test)
```

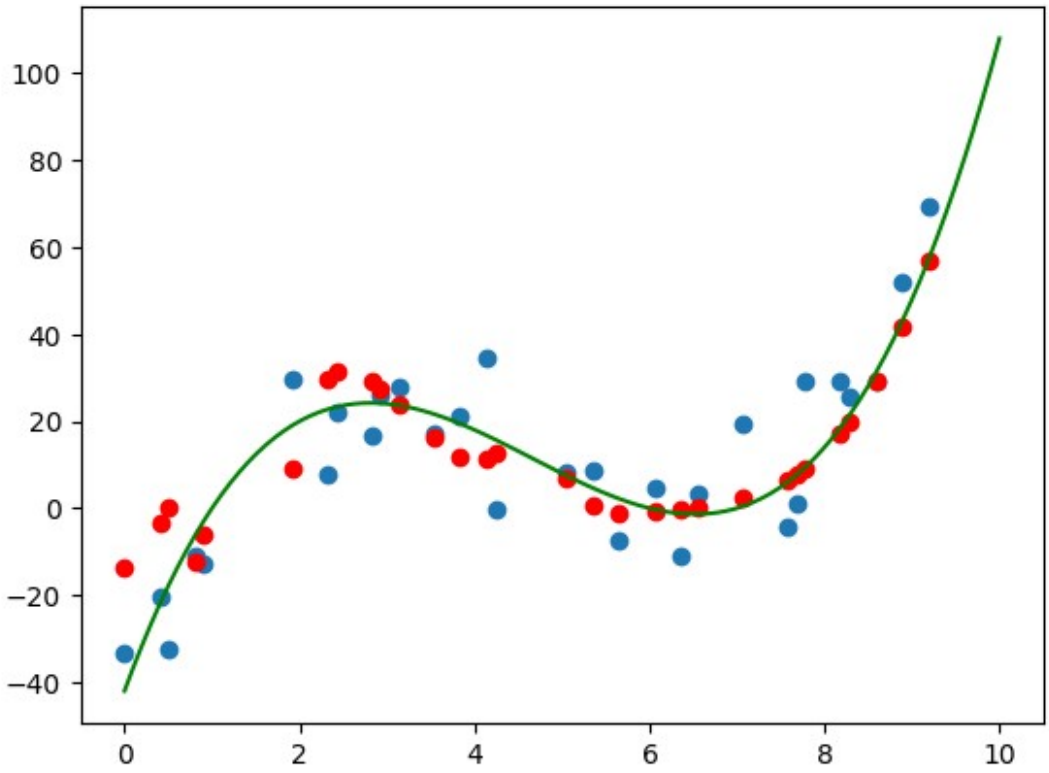
See also: https://stackoverflow.com/questions/60018578/what-does-model-eval-do-in-pytorch

```
x_train = x_train.reshape(-1, 1).astype(np.float32)
y_train = y_train.reshape(-1, 1).astype(np.float32)

x_test = x_test.reshape(-1, 1).astype(np.float32)
```

```
y_test = y_test.reshape(-1, 1).astype(np.float32)

x_tensor_train = torch.from_numpy(x_train)
y_tensor_train = torch.from_numpy(y_train)

x_tensor_test = torch.from_numpy(x_test)
y_tensor_test = torch.from_numpy(y_test)

# --- Model, optimizer, loss ---
model = SimpleRBFNN(n_h=50)
optimizer = optim.RMSprop(model.parameters())
loss_fn = nn.MSELoss()

# --- Traning ---
epochs = 10000
history = {'loss': [], 'val_loss': []}

for epoch in range(epochs):
    model.train()

    optimizer.zero_grad()
    y_pred = model(x_tensor_train)
    loss = loss_fn(y_pred, y_tensor_train)
    loss.backward()
    optimizer.step()

    model.eval()
    with torch.no_grad():
        y_val_pred = model(x_tensor_test)
        val_loss = loss_fn(y_val_pred, y_tensor_test).item()

    history['loss'].append(loss.item())
    history['val_loss'].append(val_loss)

    if (epoch+1) % 1000 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")


Epoch 1000, Loss: 143.713043
Epoch 2000, Loss: 110.495857
Epoch 3000, Loss: 107.487473
Epoch 4000, Loss: 106.831512
Epoch 5000, Loss: 106.217987
Epoch 6000, Loss: 104.177376
Epoch 7000, Loss: 99.408005
Epoch 8000, Loss: 99.262054
Epoch 9000, Loss: 97.240906
Epoch 10000, Loss: 98.701622

plt.scatter(x_test,y_test)
plt.plot(x,(x-1)*(x-6)*(x-7),color='g')
```

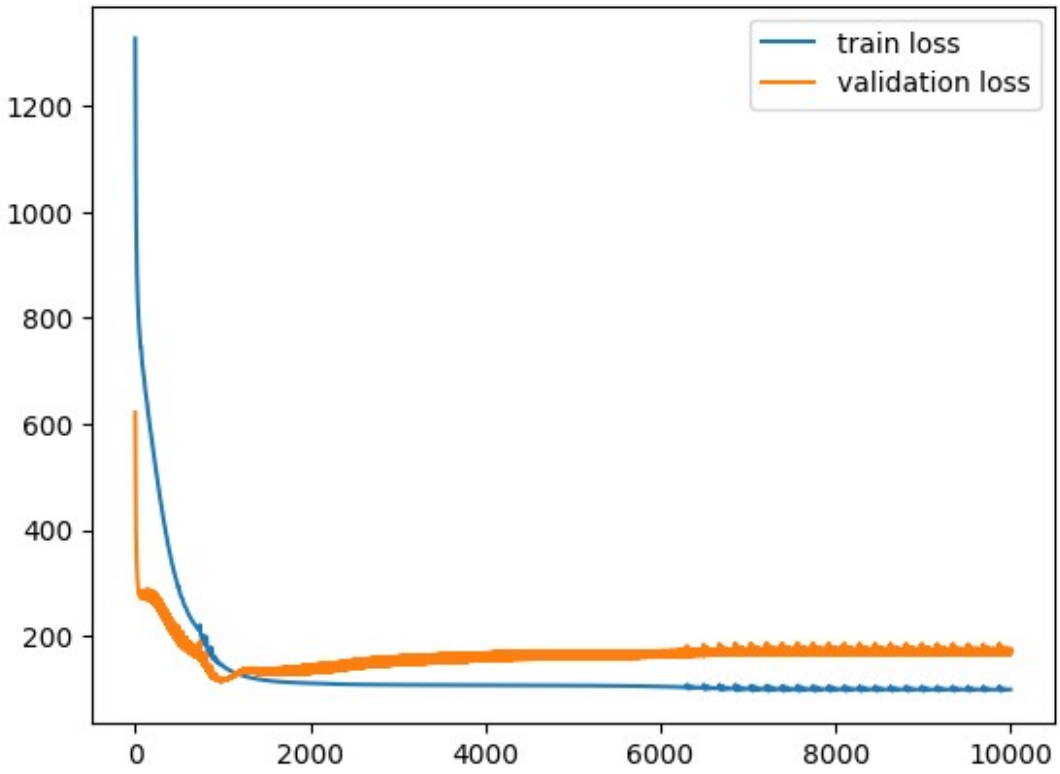
```
y_pred=model(x_tensor_test)
plt.scatter(x_test,y_pred.detach().numpy(),color='r')
# plt.plot(x_test,y_pred,color='r')
```

```
<matplotlib.collections.PathCollection at 0x7aa443b58b90>
```




```
plt.plot(history['loss'],label='train loss')
plt.plot(history['val_loss'],label='validation loss')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7aa443b5b650>
```



1.8 Classification

Function models can be used for classification, provided we constrain them to return probabilities, i.e. values from [0,1] interval.

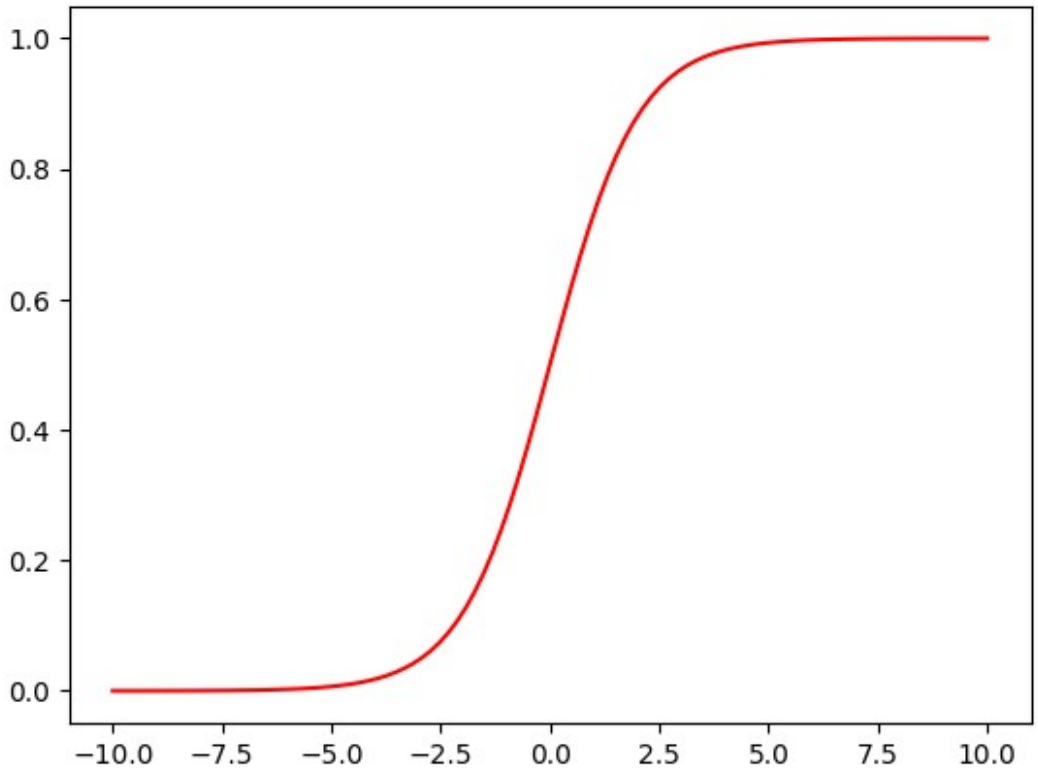
- Function with one output may be used for binary classification:
  - Assign $label_0$ if $f(x)$<0.5
  - Assign $label_1$ if $f(x) \geq 0.5$

TODO 1.8.1 Which function converts $R \rightarrow [0,1)$? Answer the question

- SIGMOID

```
import matplotlib.pyplot as plt
x=np.linspace(-10,10,100)

plt.plot(x,(lambda x: 1/(1+np.exp(-x)))(x),c='r')
```

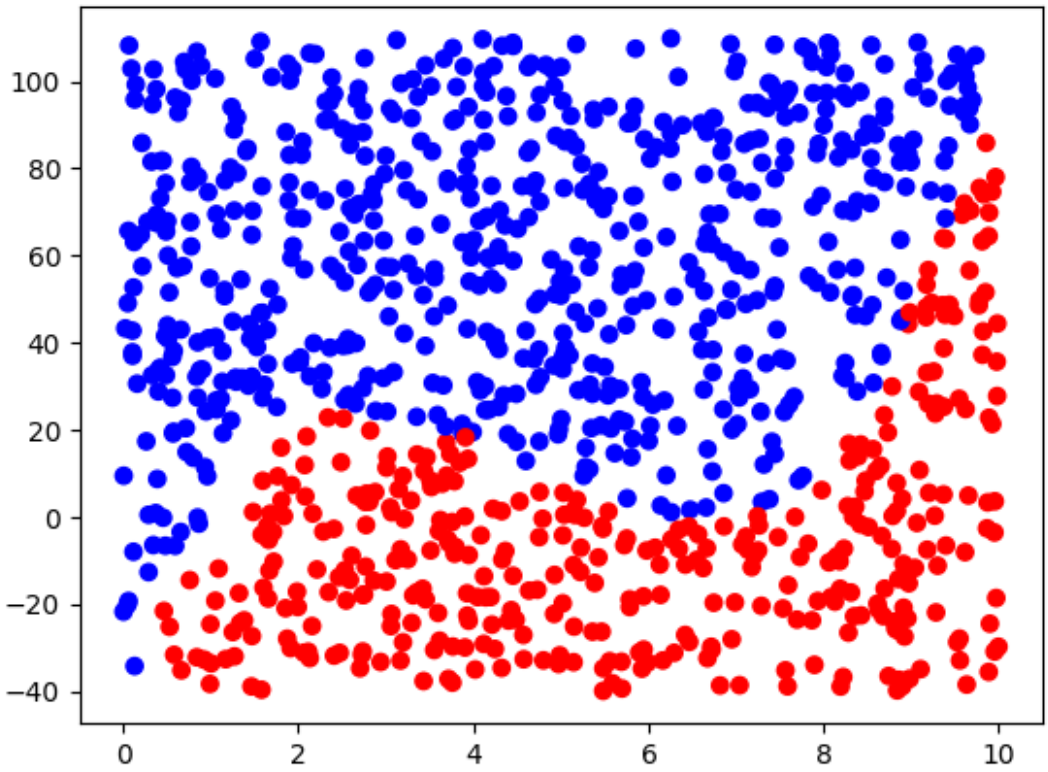
```
[<matplotlib.lines.Line2D at 0x7aa444a1bdd0>]
```



We will generate a dataset. Points above the previously used polynomial will have blue label, the points below red.

```
X = np.random.rand(1000,2)*[10,150]-[0,40]
y = np.where(X[:,1]>(X[:,0]-1)*(X[:,0]-6)*(X[:,0]-7),1,0)
# y.shape

from matplotlib.colors import ListedColormap
cm = ListedColormap(['r', 'b'])
plt.scatter(X[:,0],X[:,1],c=y,cmap=cm)
```

```
<matplotlib.collections.PathCollection at 0x7aa4466a7470>
```



We will biuld a model more suitable for classification.

What is binary_crossentropy aka. logloss?

$$loss_i = -\left[y_i \cdot ln(p_i) + (1 - y_i) \cdot ln(1 - p_i) \right]$$

You may google the term...

$$p_i = f(x_1)$$

$$y_i = 1 \rightarrow p_i = 1(.9)$$

$$y_i = 0 \rightarrow p_i = 0(.1)$$

1.8.1 Classification in TensorFlow

```
import tensorflow as tf
from tensorflow.keras import layers, models


def build_classification_model(n_h):
  model = models.Sequential()
  model.add(layers.Input(shape=(2,)))
  model.add(layers.Dense(n_h, activation='relu'))
  model.add(layers.Dense(n_h, activation='relu'))
  model.add(layers.Dense(1,activation='sigmoid'))
```

```
  model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
  return model
```

TODO 1.8.2 fit the model using training data. Set about 100 epochs, use X_test and y_test as validation data.

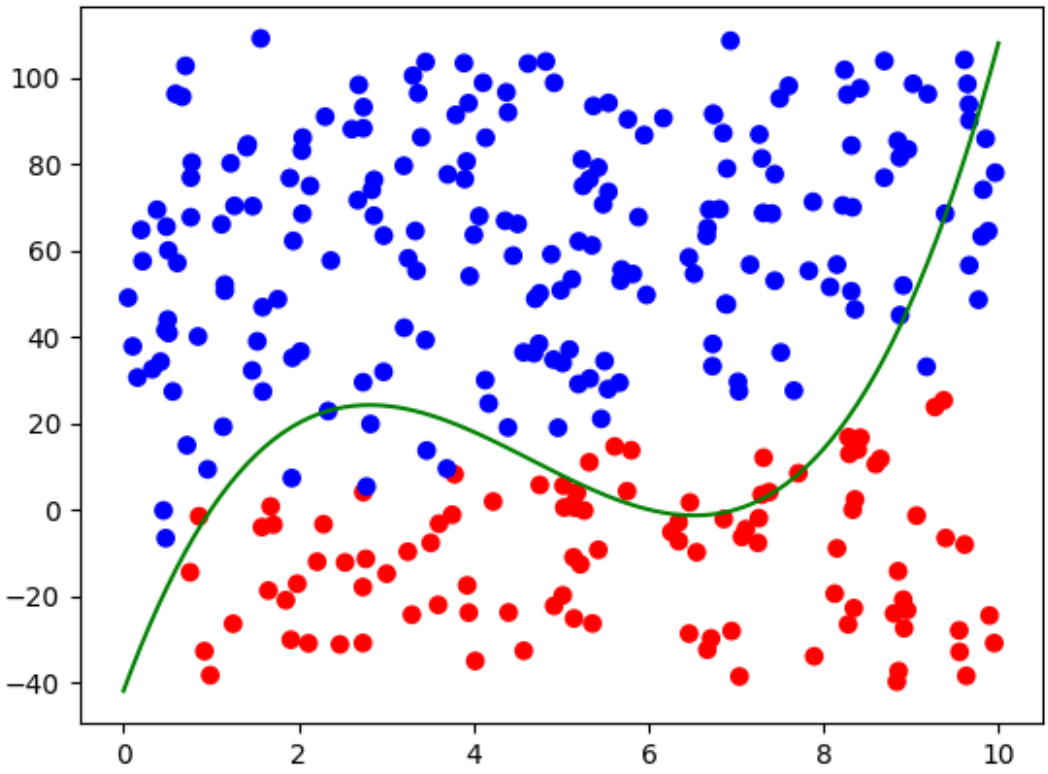
```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=123)

model=build_classification_model(32)
history=model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=0
)
```

Display predictions and the polynomial curve which was used to separate class instances.

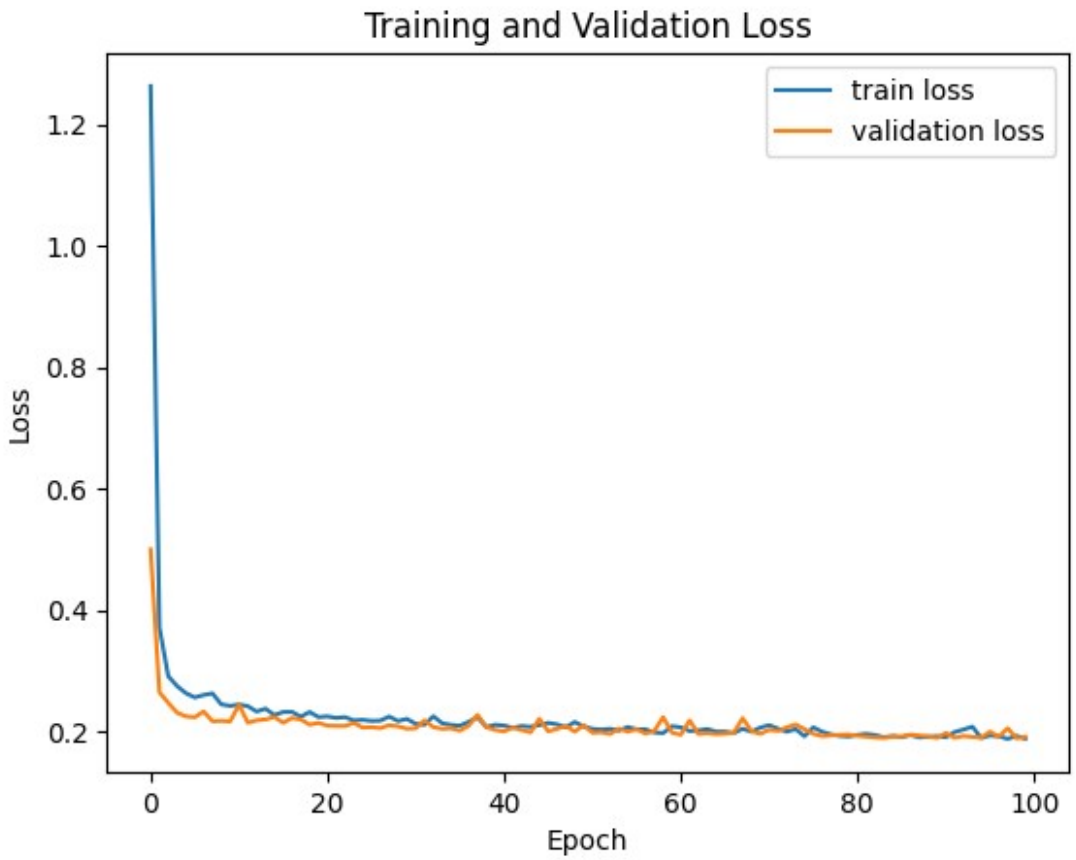
```
y_pred = model.predict(X_test)
plt.scatter(X_test[:,0],X_test[:,1],c=y_pred,cmap=cm)
x = np.linspace(0,10,100)
y = (x-1)*(x-6)*(x-7)
plt.plot(x,y,color='g')
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
```

```
[<matplotlib.lines.Line2D at 0x7aa4446a8ec0>]
```




```
plt.plot(history.history['loss'],label='train loss')
plt.plot(history.history['val_loss'],label='validation loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()
```



1.8.1 Classification in Pytorch

TODO 1.8.3 Build PyTorch model with the same architecture as TensorFlow. Split X,y dataset to train and test data. Train the model and collect loss and validation loss in the history dictionary. Display plots.

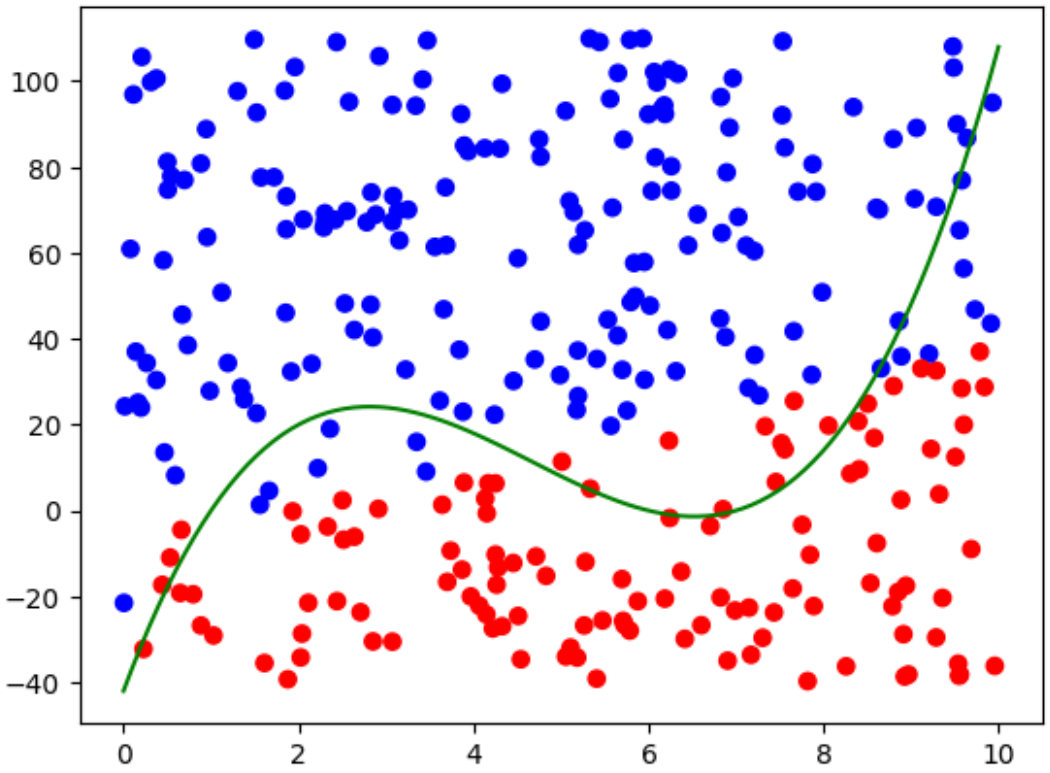
```
import torch
import torch.nn as nn
import torch.optim as optim

# --- Model ---
class ClassificationModel(nn.Module):
    def __init__(self, n_h):
        super().__init__()
        self.hidden1 = nn.Linear(2, n_h)     # Input (2) -> n_h
        self.hidden2 = nn.Linear(n_h, n_h)   # n_h -> n_h
        self.out = nn.Linear(n_h, 1)         # n_h -> 1
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden1(x))   # 1. Dense + ReLU
        x = self.relu(self.hidden2(x))   # 2. Dense + ReLU
```

```
        x = self.sigmoid(self.out(x))    # Output + Sigmoid
        return x

X = np.random.rand(1000,2)*[10,150]-[0,40]
y = np.where(X[:,1]>(X[:,0]-1)*(X[:,0]-6)*(X[:,0]-7),1,0)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=123)

X_train = torch.from_numpy(X_train).float()
y_train = torch.from_numpy(y_train).float()
X_test = torch.from_numpy(X_test).float()
y_test = torch.from_numpy(y_test).float()

# Reshape y_train and y_test to match the model output shape
y_train = y_train.reshape(-1, 1)
y_test = y_test.reshape(-1, 1)

# --- Model, optimizer, loss ---
model = ClassificationModel(n_h=32)
optimizer = optim.Adam(model.parameters(), lr = 0.005)
loss_fn = nn.BCELoss()

epochs = 100
history = {'loss': [], 'val_loss': []}

for epoch in range(epochs):
    model.train()

    optimizer.zero_grad()
    y_pred = model(X_train)
    loss = loss_fn(y_pred, y_train)
    loss.backward()
    optimizer.step()

    model.eval()
    with torch.no_grad():
        y_val_pred = model(X_test)
        val_loss = loss_fn(y_val_pred, y_test)

    history['loss'].append(loss.item())
    history['val_loss'].append(val_loss)

    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1}, Train Loss: {loss.item():.4f}, Val
Loss: {val_loss.item():.4f}")


Epoch 10, Train Loss: 0.3164, Val Loss: 0.2763
Epoch 20, Train Loss: 0.2861, Val Loss: 0.2513
Epoch 30, Train Loss: 0.2654, Val Loss: 0.2319
Epoch 40, Train Loss: 0.2609, Val Loss: 0.2296
```

```
Epoch 50, Train Loss: 0.2548, Val Loss: 0.2244
Epoch 60, Train Loss: 0.2503, Val Loss: 0.2209
Epoch 70, Train Loss: 0.2460, Val Loss: 0.2180
Epoch 80, Train Loss: 0.2422, Val Loss: 0.2153
Epoch 90, Train Loss: 0.2380, Val Loss: 0.2128
Epoch 100, Train Loss: 0.2338, Val Loss: 0.2102
```

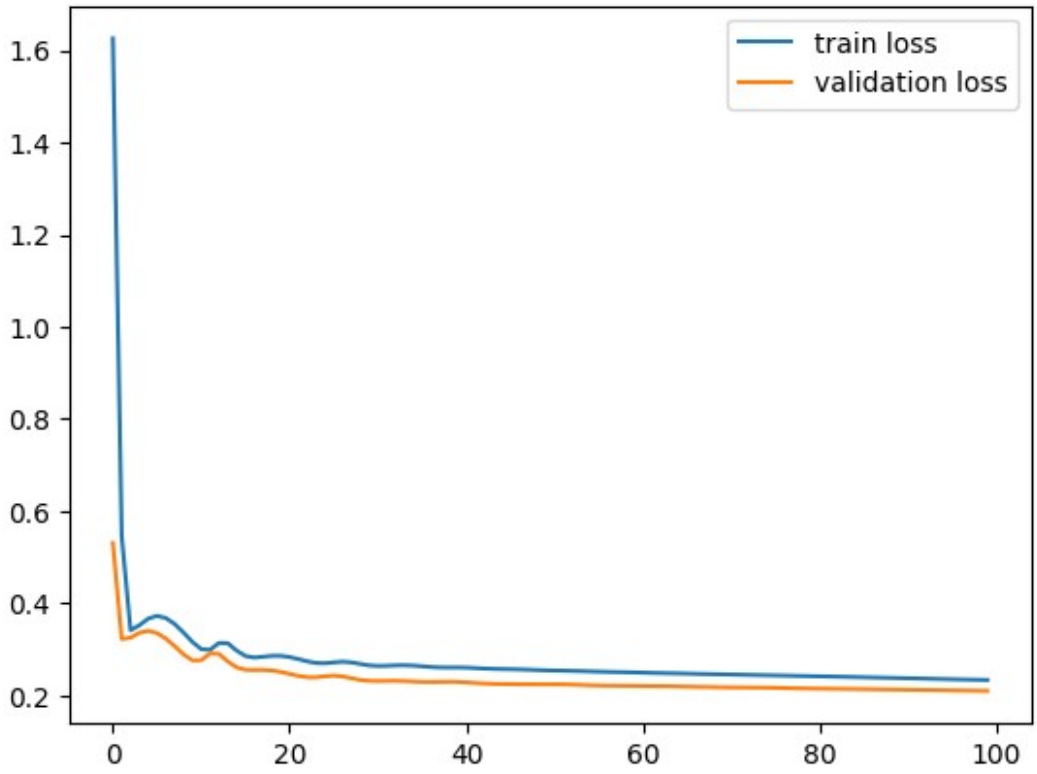
```
y_pred = model(X_test)
plt.scatter(X_test[:,0],X_test[:,1],c=y_pred.detach().numpy(),cmap=cm)
x = np.linspace(0,10,100)
y = (x-1)*(x-6)*(x-7)
plt.plot(x,y,color='g')
```

```
[<matplotlib.lines.Line2D at 0x7aa43c674890>]
```




```
plt.plot(history['loss'],label='train loss')
plt.plot(history['val_loss'],label='validation loss')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7aa43c69f530>
```


train loss
validation loss
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0
20
40
60
80
100